

IE437 Data-Driven Decision Making and Control
2023 Spring Semester

8. Value Based Reinforcement Learning

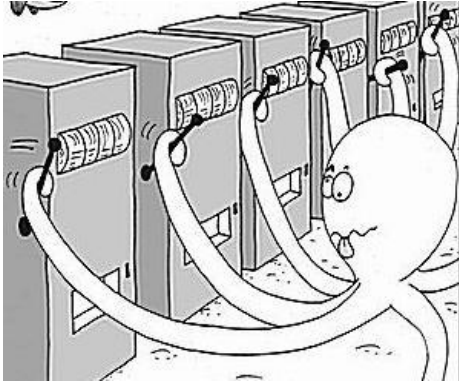
Jinkyoo Park
Associate Professor
Industrial and Systems Engineering Department

Value-Based Reinforcement Learning

1. Exploitation vs Exploration
2. Monte Carlo method
3. Temporal Different method
4. Deep Q-Learning

1. Exploitation vs Exploration

An n -Armed Bandit Problem



- There are n machine
- Each machine i returns a reward $r \sim P(\theta_i)$: θ_i is unknown
- $a_t \in \{1, \dots, n\}$: the choice of machine at time t
- r_t : the reward at time t
- Policy π maps all the history to new action:

$$\pi: [(a_1, r_1), (a_2, r_2), \dots, (a_{t-1}, r_{t-1})] \rightarrow a_t$$

Find the optimal policy π^* that maximizes $E[\sum_{t=1}^T r_t]$ or $E[r_T]$

Acquiring new information
(exploration)

trade-off

capitalizing on the information
available so far
(exploitation)

Internet add : Advertising showing strategy for users

Finance : Portfolio optimization under unknown return profiles (risk vs mean profit)

Health care : Choosing the best treatment among alternatives

Internet shopping : Choosing the optimum price (sales v.s. profits)

Experiment design : Sequential experimental design (or sequential simulation parameter)

An n -Armed Bandit Problem

The value of the action $Q^*(a)$:

The true value of action a is defined as the mean reward received when that action is selected

$$Q^*(a) = E[r]$$

At any time there will be at least one action whose **value of the action** is the largest

- **Select greedy action (Exploiting)**: the action with the largest value of action
- **Select non-greedy action (Exploring)**: the action enabling you to improve your estimate of the non-greedy actions' value

Exploiting

Go to a company to make money right after undergraduate



Exploring

Go to a graduate school to explore my intellectual capability?



How to balance between exploitation and exploration ?

Action-Value Method

- Action-value method estimates the value of an action and use this to select the action
- Natural way to estimate the *action-value* is by averaging the rewards actually received when the action was selected
- If at time t , **action a has been chosen k_a times prior to t** , yielding rewards, $r_1, r_2, \dots, r_{k_a}$, the estimated action value $Q_t(a)$ for a at time t is defined as

$$Q_t(a) = \frac{r_1 + r_2 + \dots + r_{k_a}}{k_a}$$

If $k_a = 0$, $Q_t(a) = 0$
If $k_a = \infty$, $Q_t(a) \rightarrow Q^*(a)$

- Greedy action selection rule is

$$a_t = \operatorname{argmax}_a Q_t(a)$$

- ✓ Always exploits current knowledge to maximize immediate reward, that is it spends no time at all sampling apparently inferior actions to see if they might really be better

Action-Value Method

- If at t th play, action a has been chosen k_a times prior to t , yielding rewards, $r_1, r_2, \dots, r_{k_a}$, the estimated action value for a is defined as

$$Q_t(a) = \frac{r_1 + r_2 + \dots + r_{k_a}}{k_a}$$

If $k_a = 0$, $Q_t(a) = 0$
If $k_a = \infty$, $Q_t(a) \rightarrow Q^*(a)$

greedy action selection rule

$$a_t = \underset{a}{\operatorname{argmax}} Q_t(a)$$

- ✓ Always exploits current knowledge to maximize immediate reward
- ✓ spends no time at all sampling apparently inferior actions to see if they might really be better

ϵ - *greedy* action selection rule

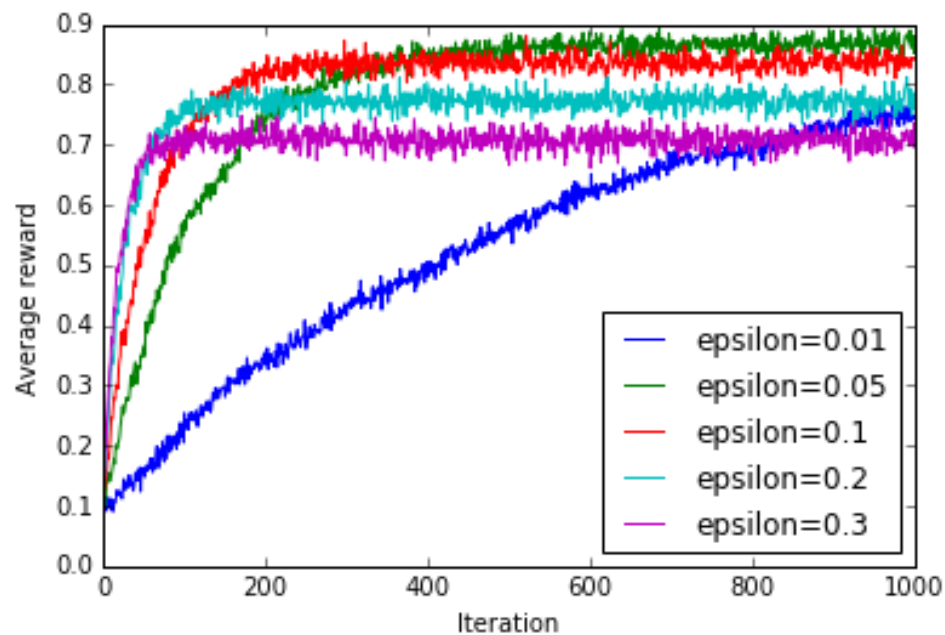
$$\pi(a) = \begin{cases} 1 - \epsilon + \epsilon/|A(s)| & \text{If } a = a^* \\ \epsilon/|A(s)| & \text{If } a \neq a^* \end{cases}$$

$$\left(1 - \epsilon + \frac{\epsilon}{|A(s)|}\right) \times 1 + \frac{\epsilon}{|A(s)|} (|A(s)| - 1) = 1$$

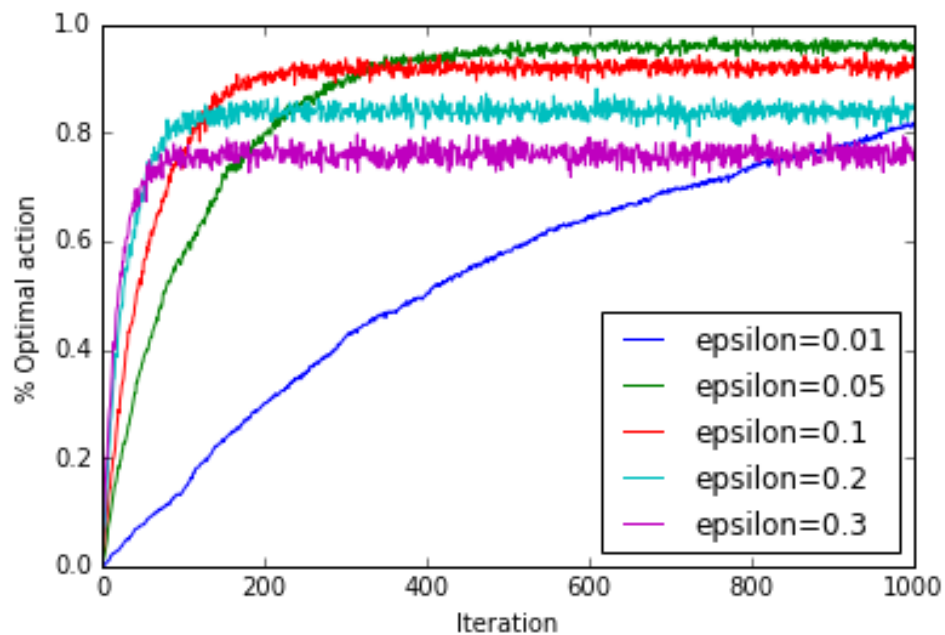
- ✓ In the limit as the number of plays increases, every a will be sampled an infinite number of times, guaranteeing $k_a \rightarrow \infty$ thus $Q_t(a) \rightarrow Q^*(a)$
- ✓ The probability of selecting the optimum action converges to greater than $1 - \epsilon$

Action-Value Method

Average reward $E[r_t]$



% of selecting a^*



Relationship with model based approach

		Only Exploitation	Exploration vs Exploitation
Optimum action	Single state	Model is known Optimization	Model is unknown Bandit
	Multiple state	Dynamic Programing	Contextual Bandit Reinforcement learning

- We going to learn Dynamic Programming approach first, and move to Reinforcement learning

From MDP to Reinforcement Learning



Markov Decision Process (Offline)

- Have mental model of how the world works
- Find policy to collect the maximum rewards

$$\text{Solve } Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$
$$\text{Find } \pi^*(s) = \max_a Q^*(s, a)$$



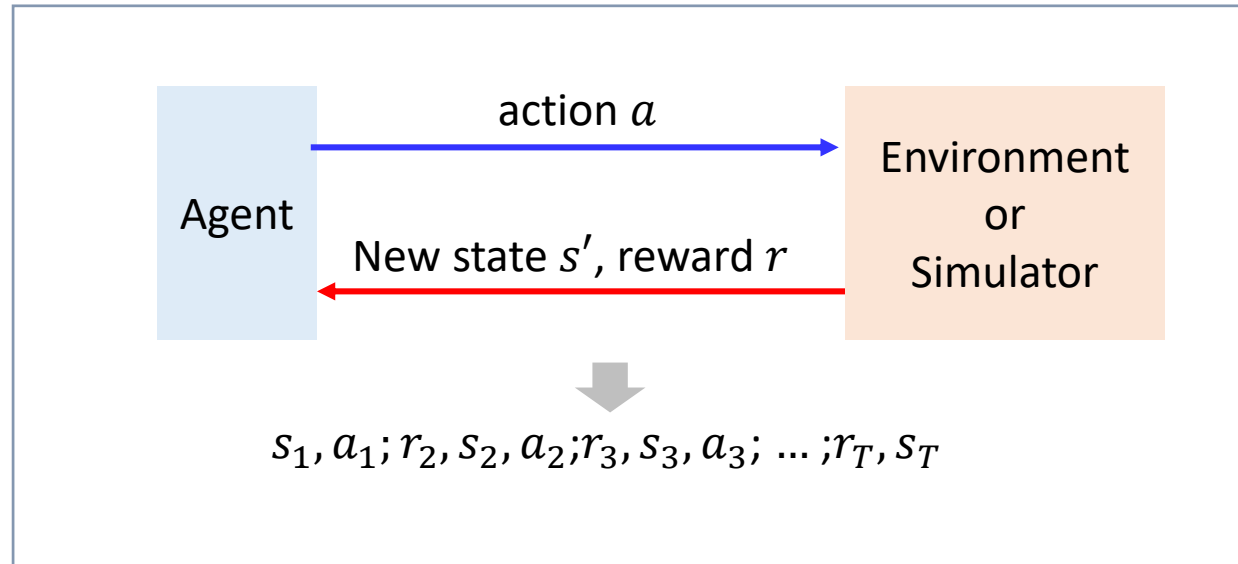
Reinforcement Learning (Offline & Online)

- Don't know how the world works
- Perform a sequence of actions in the world to maximize the rewards

$$s_1, a_1, r_1; s_2, a_2, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T \rightarrow Q^*(s, a) \rightarrow \pi^*(s)$$

- Reinforcement learning is really the way humans work:
 - we go through life, taking various actions, getting feedback.
 - We get rewarded for doing well and learn along the way.

Reinforcement Learning Template



Template for Reinforcement Learning

For $t = 1, 2, 3, \dots$

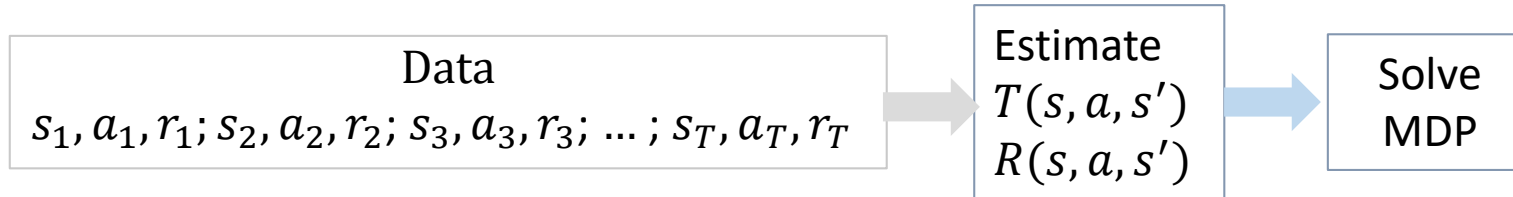
Choose action $a_t = \pi(s_t)$ (how?) : **Decision making**

Receive reward r_{t+1} and observe new state s_{t+1} (Environment)

Update parameters associated with $V(t)$, $Q(s, a)$ (how?) : **Learning**

Road Map

- Model-Based Reinforcement learning



- Model-FREE Reinforcement learning

How to estimate $V^*(s)$ and $Q^*(s, a)$

Monte Carlo method

Temporal Difference methods

		Non-Bootstrap	Bootstrap
How to explore ?	On-policy	On-policy Monte Carlo Control	SARSA
	Off-policy	Off-policy Monte Carlo Control	Q-Learning

- Episodic based

- Single-data-point based

2. Monte Carlo Based Learning

Model-Based Reinforcement Learning (Monte Carlo Methods)

Key Idea : model-based learning

- Formulate the MDP using the estimated $T(s, a, s')$ and $R(s, a, s')$
- Solve the MDP using various solution methods, i.e., DP approach

$$\begin{aligned} \text{Solve } Q^*(s, a) &= \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')] \\ \text{Find } \pi^*(s) &= \max_a Q^*(s, a) \end{aligned}$$

- Data following policy π

$$s_1, a_1, r_1; s_2, a_2, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T$$

- Transition model

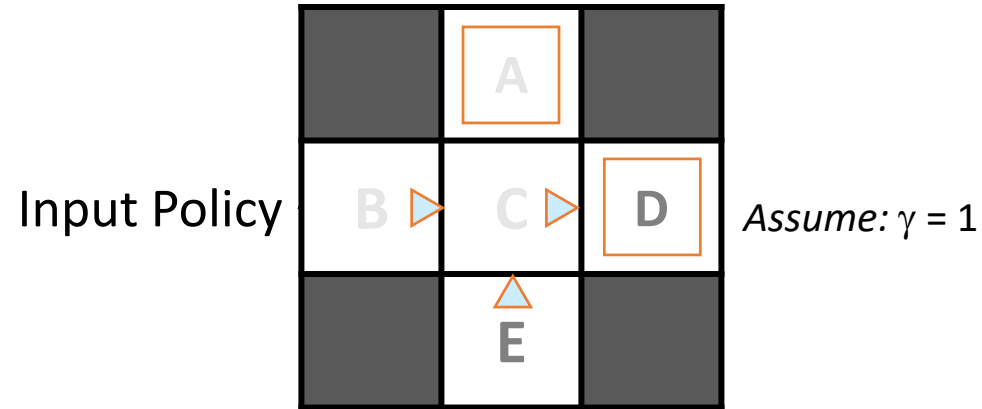
$$\hat{T}(s, a, s') = \frac{\text{\#times } (s, a, s') \text{ occurs}}{\text{\#times } (s, a) \text{ occurs}}$$

- Reward model

$$\hat{R}(s, a, s') = \text{average of } r \text{ in } (s, a, r, s')$$

Model-Based Reinforcement Learning (Monte Carlo Methods)

Example



- Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

- Learned Model

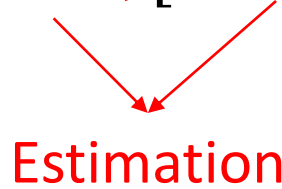
$$T(s, a, s')$$

$T(B, \text{east}, C) = 1.00$
 $T(C, \text{east}, D) = 0.75$
 $T(C, \text{east}, A) = 0.25$

$$R(s, a, s')$$

$R(B, \text{east}, C) = -1$
 $R(C, \text{east}, D) = -1$
 $R(D, \text{exit}, x) = +10$

From Model-Based to Model Free

$$Q^*(s, a) = \sum_{s'} \hat{T}(s, a, s') [\hat{R}(s, a, s') + \gamma V^*(s')]$$


Estimation

Why not estimating $Q^*(s, a)$ directly
instead of separately estimating $\hat{T}(s, a, s')$ and $\hat{R}(s, a, s')$?

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

Model-Free Monte Carlo Based Methods

		How to estimate $V^*(s)$ and $Q^*(s, a)$	
		Monte Carlo method	Temporal Difference methods
How to explore ?		Non-Bootstrap	Bootstrap
	On-policy	On-policy Monte Carlo Control	SARSA
	Off-policy	Off-policy Monte Carlo Control	Q-Learning

- Episodic based
- Single-data-point based

Model-Free Monte Carlo Based Methods

- Monte Carlo methods require only experience-sample sequences of states, actions, and rewards from on-line or simulated interaction with an environment
- Monte Carlo methods are ways of solving the reinforcement learning problem based on **averaging sample returns (Monte Carlo)**

$$R(s, a) = \text{average of } r \text{ in } (s, a, r)$$

- Monte Carlo methods are incremental in an **episode-by-episode sense**, but not in a step-by-step sense
→ after a final state has reached, reward appears, e.g., Go

Dynamic Programming

- Policy Evaluation
- Policy Iteration
- Policy Improvement

Key ideas

Monte Carlo Methods

- Policy Evaluation
- Policy Iteration
- Policy Improvement

Monte Carlo Policy Evaluation

Key Idea : Monte Carlo Policy Evaluation

- Learn the state-value function $V^\pi(s)$ for a given policy π
- The value of state is the expected utility - **expected accumulative future reward** starting from s and following the policy π

Data (following policy π):

$$s_t \in \mathcal{S} = \{s^1, s^2, s^3\}, a_t \in \mathcal{A} = \{a^1, a^2, a^3\}$$

Episode 1: $s_1, a_1, r_1; s_2, a_2, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T$

Episode 2: $s_1, a_1, r_1; s_2, a_2, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T$

Episode 3: $s_1, a_1, r_1; s_2, a_2, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T$

$\vdots$

Utility u_t :

$$u_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

$$u_t = r_t + r_{t+2} + \dots + r_T$$

Estimate $V^\pi(s)$ or $Q^\pi(s, a)$:

$$V^\pi(s) = \text{average of } u_t \text{ where } s_t = s \text{ and following } \pi$$

Because the value being computed is dependent on the policy used to generate the data, we call this an **on-policy** algorithm.

Monte Carlo Policy Evaluation

Key Idea : Monte Carlo Policy Evaluation

- Learn the state-value function $V^\pi(s)$ for a given policy π
- The value of state is the expected utility - **expected accumulative future reward** starting from s and following the policy π

$$s_t \in \mathcal{S} = \{s^1, s^2, s^3\}, a_t \in \mathcal{A} = \{a^1, a^2, a^3\}$$

(State, Action, Reward) pairs generated by policy π

Episode 1: ($s^1, a^2, 1$); ($s^3, a^1, 5$); ($s^2, a^3, 3$), ($s^1, a^3, 10$), ($s^2, a^2, 2$)

Episode 2: ($s^3, a^1, 5$); ($s^2, a^2, 2$); ($s^1, a^2, 1$); ($s^2, a^3, 3$), ($s^1, a^3, 10$)

Episode 3: ($s^2, a^3, 3$); ($s^1, a^2, 1$); ($s^3, a^1, 5$); ($s^1, a^3, 10$), ($s^2, a^2, 2$)

($\gamma = 1$)

Episode 1: $V^\pi(s^1) = 1 + 5 + 3 + 10 + 2 = 21$

Episode 2: $V^\pi(s^1) = 1 + 3 + 10 = 14$

Episode 3: $V^\pi(s^1) = 1 + 5 + 10 + 2 = 19$

First visit to s

$V^\pi(s^1) =$ average of accumulated reward over all episodes

$$= \frac{21+14+19}{3} = 14.6$$

Monte Carlo Policy Evaluation

Key Idea : Monte Carlo Policy Evaluation

- Learn the state-value function $V^\pi(s)$ for a given policy π
- The value of state is the expected utility - **expected accumulative future reward** starting from s and following the policy π

$$s_t \in \mathcal{S} = \{s^1, s^2, s^3\}, a_t \in \mathcal{A} = \{a^1, a^2, a^3\}$$

(State, Action, Reward) pairs generated by policy π

Episode 1: ($s^1, a^2, 1$); ($s^3, a^1, 5$); ($s^2, a^3, 3$), ($s^1, a^3, 10$), ($s^2, a^2, 2$)

Episode 2: ($s^3, a^1, 5$); ($s^2, a^2, 2$); ($s^1, a^2, 1$); ($s^2, a^3, 3$), ($s^1, a^3, 10$)

Episode 3: ($s^2, a^3, 3$); ($s^1, a^2, 1$); ($s^3, a^1, 5$); ($s^1, a^3, 10$), ($s^2, a^2, 2$)

($\gamma = 1$)

Episode 1: $V^\pi(s^1) = 1 + 5 + 3 + 10 + 2 = 21$; $V^\pi(s^1) = 10 + 2 = 12$

Episode 2: $V^\pi(s^1) = 1 + 3 + 10 = 14$; $V^\pi(s^1) = 10 = 10$

Episode 3: $V^\pi(s^1) = 1 + 5 + 10 + 2 = 19$; $V^\pi(s^1) = 10 + 2 = 12$

Every visit to s

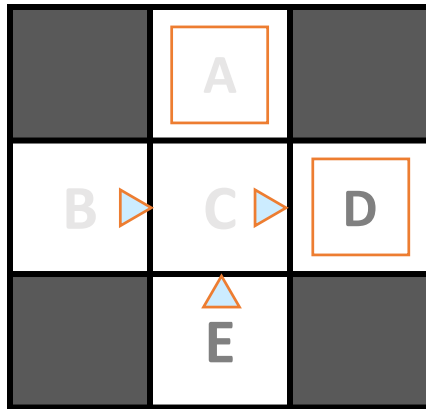
$V^\pi(s^1)$ = average of accumulated reward over all episodes

$$= \frac{21+12+14+10+19+12}{6} = 15$$

Monte Carlo Policy Evaluation

Example

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Output Values

	-10	
+8	+4	+10
	-2	

Monte Carlo Policy Evaluation

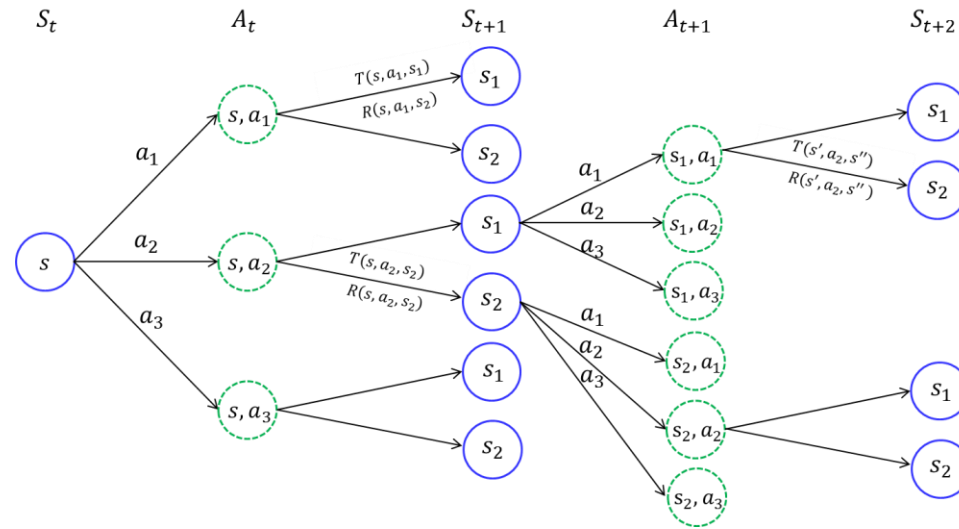
Example

- What's good about direct evaluation?
 - It's easy to understand
 - It doesn't require any knowledge of T , R
 - It eventually computes the correct average values, using just sample transitions
- What bad about it?
 - It wastes information about state connections
 - Each state must be learned separately
 - So, it takes a long time to learn

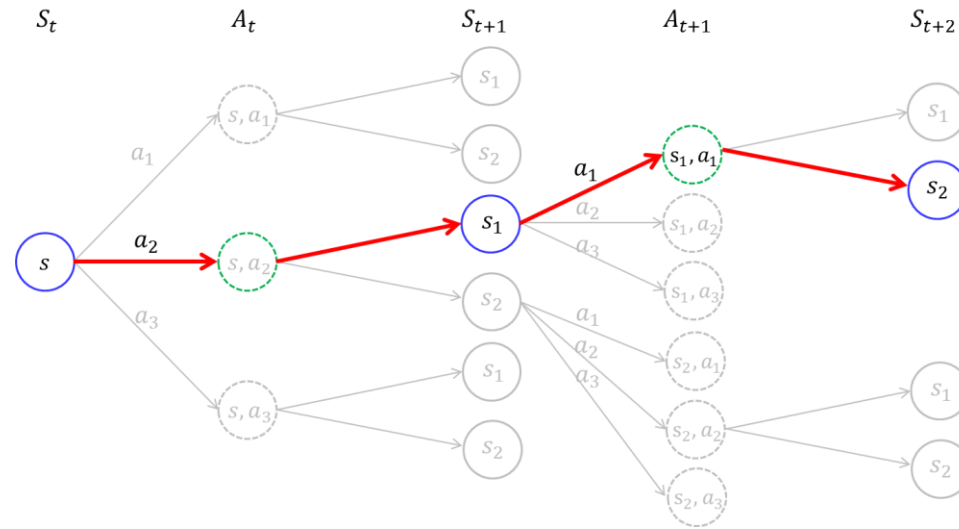
Output Values

	-10 A	
+8 B	+4 C	+10 D
	-2 E	

Monte Carlo Policy Evaluation



Monte Carlo Policy Evaluation



- Estimates for each state are independent
 - ✓ The estimate for one state does not build upon the estimate of any other state (No bootstrap)
- The computational expense of estimating the value of a single state is independent of the number of states
 - ✓ Attractive when one requires the value of only a subset of the states

Monte Carlo Estimation of Action Values

- Without a model, state value function $V^\pi(s)$ is not enough
 - ✓ We need $T(s, a, s')$ and $R(s, a, s')$ to compute the optimum policy:

$$Q^*(s, a) = \sum_{s'} T(s, a, s') \{R(s, a, s') + \gamma V^*(s')\}$$

$$a^* = \pi^*(s) = \operatorname{argmax}_{a \in \mathcal{A}(s)} Q^*(s, a)$$

- One of primary goals for Monte Carlo method is to estimate $Q^*(s, a)$

Monte Carlo Estimation of Action Values

Key Idea : Monte Carlo Policy Evaluation

- Learn the action-value function $Q^\pi(s, a)$ for a given policy π
- The value of action-state is the expected utility - **expected accumulative future reward** starting from s and following the policy π

Data (following policy π):

$$s_t \in \mathcal{S} = \{s^1, s^2, s^3\}, a_t \in \mathcal{A} = \{a^1, a^2, a^3\}$$

Episode 1: $s_1, a_1, r_1; s_2, a_2, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T$

Episode 2: $s_1, a_1, r_1; s_2, a_2, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T$

Episode 3: $s_1, a_1, r_1; s_2, a_2, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T$

$\vdots$

Utility u_t :

$$u_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

$$u_t = r_t + r_{t+2} + \dots + r_T$$

Estimate $Q^\pi(s, a)$:

$$Q^\pi(s, a) = \text{average of } u_t \text{ where } s_t = s, a_t = a \text{ and following } \pi$$

Monte Carlo Policy Evaluation

Key Idea : Monte Carlo Policy Evaluation

- Learn the state-value function $Q^\pi(s, a)$ for a given policy π
- The value of action-state is the expected utility - **expected accumulative future reward** starting from s and following the policy π

$$s_t \in \mathcal{S} = \{s^1, s^2, s^3\}, a_t \in \mathcal{A} = \{a^1, a^2, a^3\}$$

(State, Action, Reward) pairs generated by policy π

Episode 1: ($s^1, a^2, 1$); ($s^3, a^1, 5$); ($s^2, a^3, 3$), ($s^1, a^3, 10$), ($s^2, a^2, 2$)

Episode 2: ($s^1, a^1, 5$); ($s^2, a^2, 2$); ($s^1, a^2, 1$); ($s^2, a^3, 3$), ($s^1, a^3, 10$)

Episode 3: ($s^2, a^3, 3$); ($s^1, a^2, 1$); ($s^3, a^1, 5$); ($s^1, a^3, 10$), ($s^2, a^2, 2$)

($\gamma = 1$)

Episode 1: $Q^\pi(s^1, a^2) = 1 + 5 + 3 + 10 + 2 = 21$

Episode 2: $Q^\pi(s^1, a^2) = 1 + 3 + 10 = 14$

Episode 3: $Q^\pi(s^1, a^2) = 1 + 5 + 10 + 2 = 19$

First visit to s

$Q^\pi(s^1, a^2) =$ average of accumulated reward over all episodes

$$= \frac{21+14+19}{3} = 14.6$$

Incremental Formulation

$s_1, a_1, r_1; s_2 = s, a_2 = a, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T$
 $s_1 = s, a_1 = a, r_1; s_2, a_2, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T$
 $s_1, a_1, r_1; s_2, a_2, r_2; s_3 = s, a_3 = a, r_3; \dots; s_T, a_T, r_T$
 $\vdots$
 $s_1, a_1, r_1; s_2 = s, a_2 = a, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T$

Episode 1

Episode 2

Episode 3

Episode ∞

For each episode,

We can get state and action pair

(s, a) and the following accumulated
reward (utility) u : $(s, a) \rightarrow u$

Incremental formulation (convex combination):

On each (s, a, u) for a single episode:

$$Q^\pi(s, a) \leftarrow (1 - \eta) \underbrace{Q^\pi(s, a)}_{\text{Current estimate}} + \underbrace{\eta u}_{\text{New data}}$$

where $\eta = \frac{1}{1 + (\text{\#updates to } (s, a))}$

Stochastic gradient form :

On each (s, a, u) for a single episode:

$$Q^\pi(s, a) \leftarrow Q^\pi(s, a) - \eta(Q^\pi(s, a) - u)$$

The incremental formulations can be thought of as a way to solve the following least square estimation in an online manner

$$\min_{Q^\pi} \sum_{(s, a, u)} (Q^\pi(s, a) - u)^2$$

Issue of computing $Q^\pi(s, a)$

- If π is a deterministic policy, many relevant state –action pairs may never be visited
 - ✓ then in following π , one will observe returns only for one of the actions from each state, i.e., we won't even see (s, a) if a $a \neq \pi(s)$
 - ✓ To improve the policy π , we need to **compare** the state-action values of all the possible actions, $Q^\pi(s, a_1), Q^\pi(s, a_2), \dots$
- To estimate $Q^\pi(s, a)$ reliably, we need a large number of episodes
 - ✓ *Infinite* number of episodes are required for accurate estimations

How to resolve?

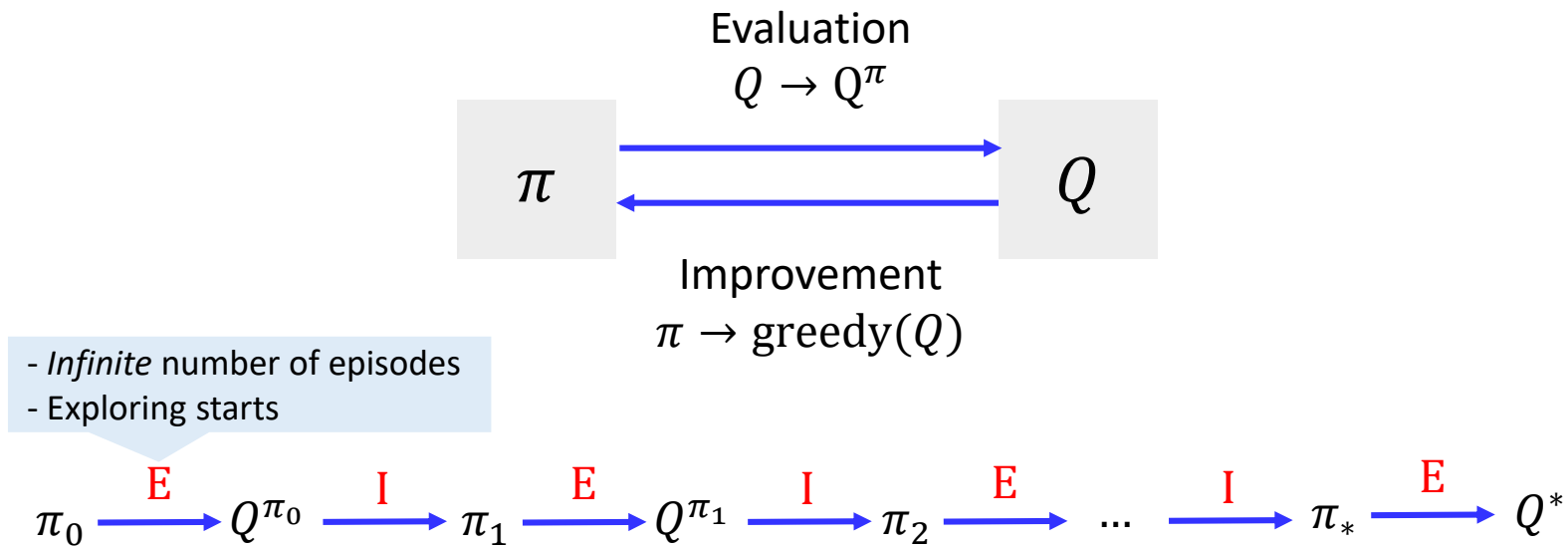
- Exploring starts + infinite number of episodes
 - ✓ Start from an arbitrary state-action pair (s, a) with a non-zero probability
 - ✓ Guarantees **that all state-action pair will be visited an infinite number of times in the limit of an infinite number of episodes**
 - ✓ Cannot be used when learning directly from experience
- Stochastic Policy + infinite number of episodes
 - ✓ Policy with **a nonzero probability of selecting all actions**
 - ✓ Guarantees that all state-action pair will be visited an infinite number of times in the limit of an infinite number of episodes

We need an efficient exploration strategy!

Monte Carlo Control

Key Idea : Monte Carlo Control

- Idea of generalized policy iteration (GPI)
- Monte Carlo Policy Evaluation + Policy improvement



- **Policy Evaluation**
 - ✓ The value function is repeatedly altered to more closely approximate the value function for the current policy π
- **Policy Improvement**
 - ✓ The policy is repeatedly improved with respect to the current action value function Q

Issue of computing $Q^\pi(s, a)$

$$\pi_0 \xrightarrow{E} Q^{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} Q^{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \dots \xrightarrow{I} \pi_* \xrightarrow{E} Q^*$$

E Policy evaluation:

- The value function is repeatedly altered to more closely approximate the value function for the current policy π
 - Infinite number of episodes under policy π
 - The episodes are generated with exploring starts
- } Converge to exact $Q^\pi(s, a)$

exploring starts

$s_1, a_1, r_1; s_2, a_2, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T \rightarrow u$	Episode 1
$s_1, a_1, r_1; s_2, a_2, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T \rightarrow u$	Episode 2
$s_1, a_1, r_1; s_2, a_2, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T \rightarrow u$	Episode 3
$s_1, a_1, r_1; s_2, a_2, r_2; s_3, a_3, r_3; \dots; s_T, a_T, r_T \rightarrow u$	Episode ∞

Average expected reward

I Policy Improvement:

- Policy improvement can be done by constructing each π_{k+1} as the greedy policy with respect to Q^{π_k}

$$\pi_{k+1}(s) = \operatorname{argmax}_a Q^{\pi_k}(s, a)$$

$$\begin{aligned} Q^{\pi_k}(s, \pi_{k+1}(s)) &= Q^{\pi_k}\left(s, \operatorname{argmax}_a Q^{\pi_k}(s, a)\right) \\ &= \max_a Q^{\pi_k}(s, a) \\ &\geq Q^{\pi_k}(s, \pi_k(s)) \\ &= V^{\pi_k}(s) \end{aligned}$$

$V^{\pi_{k+1}}(s) \geq V^{\pi_k}(s)$ for all s
(Policy improvement theorem)

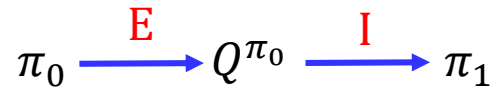
Overall process converges to an optimal policy and the optimal value function

Make Monte Carlo Control Practical

Assumptions:

- *Infinite* number of episodes
- Exploring starts

➡ **Relax** these two assumptions



- **Relaxing “Infinite number of episodes”**
 - ✓ Policy improvement with an early stop (Generalized policy improvement concept)
 - ✓ The “in place” version of value iteration
- Relaxing “exploring starts”
 - ✓ Discussed later

Monte Carlo control algorithm assuming exploring starts

Algorithm : Monte Carlo ES Control

Initialize, for all $s \in S, a \in A(s)$

$Q(s, a) \leftarrow$ arbitrary

$\pi(s) \leftarrow$ arbitrary

$U(s, a) \leftarrow$ empty list

Repeat forever:

(a) Generate *an episode* using *exploring starts* and π

(b) For each pair (s, a) appearing in the episode:

$U \leftarrow$ utility following the first occurrence of s, a

Append U to $U(s, a)$

$Q(s, a) \leftarrow \text{average}(U(s, a))$

} Policy evaluation

(c) For each s in the episode:

$\pi(s) \leftarrow \underset{a}{\operatorname{argmax}} Q(s, a)$

} Policy improvement

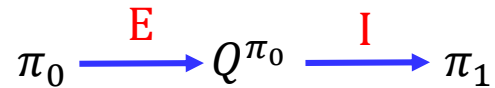
In an single episode, both Policy evaluation and policy improvement proceeds together

Make Monte Carlo Control Practical

Assumptions:

- *Infinite* number of episodes
- Exploring starts

➡ **Relax** these two assumptions



- Relaxing “Infinite number of episodes”
 - ✓ Policy improvement with an early stop (Generalized policy improvement concept)
 - ✓ The “in place” version of value iteration
- **Relaxing “exploring starts”**
 - ✓ The only general way to ensure that all actions are selected infinitely often is for the agent to continue to select them
 - ❖ On-policy methods
 - ❖ Off-policy methods

On-Policy vs Off-Policy

On policy algorithm



- On-policy methods attempt to evaluate or improve the policy that is used to make decisions (generate data)
 - ✓ Policy is soft: $\pi(s, a) > 0$ for all $s \in S$ and $a \in A(s)$

Off policy algorithm



- Behavioral policy is used to generate behavior
- Estimation policy is evaluated and improved
 - ✓ Estimation policy may be deterministic, while the behavior policy can continue to sample all possible actions

On-policy Monte Carlo Control

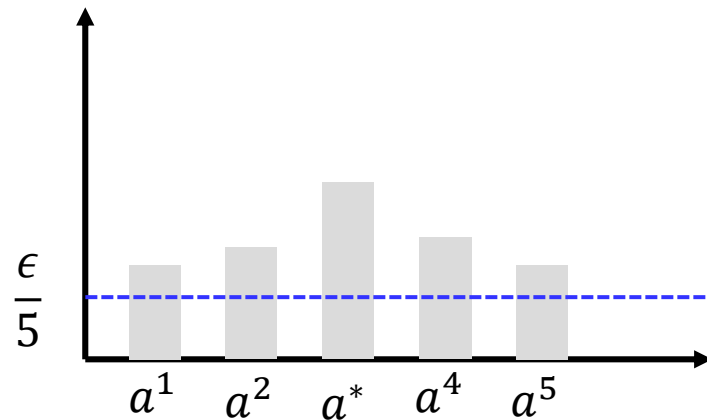
Key Idea : On-policy methods: Soft policies

- attempt to evaluate or improve the policy that is used to make decisions
- $\pi(s, a) > 0$ for all $s \in S$ and $a \in A(s)$

ϵ – soft policy

$$\pi(s, a) \geq \frac{\epsilon}{|A(s)|} \text{ for all } a$$

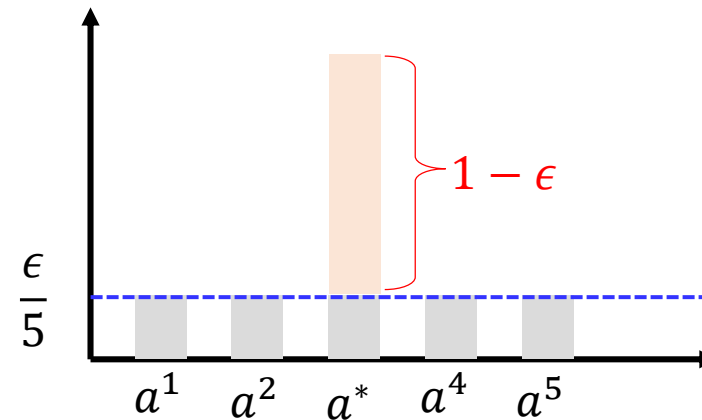
$$\sum_a \pi(s, a) = 1$$



ϵ – greedy policy

$$\pi(s, a) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A(s)|} & \text{If } a = a^* \\ \frac{\epsilon}{|A(s)|} & \text{If } a \neq a^* \end{cases}$$

$$\text{Total probability} = \left(1 - \epsilon + \frac{\epsilon}{|A(s)|}\right) 1 + \frac{\epsilon}{|A(s)|} (|A(s)| - 1) = 1$$



On-policy Monte Carlo Control

Any ϵ – *greedy* policy π' respect to Q^π is an improvement over any ϵ – *soft* policy π

Let π' be the ϵ – *greedy* policy,

$$\begin{aligned} Q^\pi(s, \pi'(s, a)) &= \sum_a \pi'(s, a) Q^\pi(s, a) \\ &= \frac{\epsilon}{|A(s)|} \sum_a Q^\pi(s, a) + (1 - \epsilon) \max_a Q^\pi(s, a) \\ &\geq \frac{\epsilon}{|A(s)|} \sum_a Q^\pi(s, a) + (1 - \epsilon) \sum_a \frac{\pi(s, a) - \frac{\epsilon}{|A(s)|}}{1 - \epsilon} Q^\pi(s, a) \\ &= \frac{\epsilon}{|A(s)|} \sum_a Q^\pi(s, a) - \frac{\epsilon}{|A(s)|} \sum_a Q^\pi(s, a) + \sum_a \pi(s, a) Q^\pi(s, a) \\ &= V^\pi(s) \end{aligned}$$

Recall Policy improvement Theorem

Policy improvement must give us a strictly better policy $\pi'(s)$ than the older policy $\pi(s)$ except when the original policy is already optimal $\pi(s, a) = \pi^*(s, a)$

$$Q^\pi(s, \pi'(s, a)) \geq V^\pi(s) \rightarrow V^{\pi'}(s) \geq V^\pi(s)$$

Thus, by the policy improvement theorem, $\pi' \geq \pi$ (i.e., $V^{\pi'}(s) \geq V^\pi(s)$ for all $s \in S$)

On-policy Monte Carlo Control

Algorithm : ϵ – *soft* On-Policy Monte Carlo Control

Initialize, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

$Q(s, a) \leftarrow$ arbitrary

$\pi \leftarrow$ an arbitrary ϵ – *soft policy*

$U(s, a) \leftarrow$ empty list

Repeat forever:

(a) Generate *an episode* using π

(b) For each pair (s, a) appearing in the episode:

$U \leftarrow$ utility following the first occurrence of s, a

Append U to $U(s, a)$

$Q(s, a) \leftarrow \text{average}(U(s, a))$

Policy evaluation

(c) For each s in the episode:

$a^* \leftarrow \underset{a \in \mathcal{A}(s)}{\operatorname{argmax}} Q(s, a)$

For all $a \in \mathcal{A}(s)$

$$\pi(s, a) = \begin{cases} 1 - \epsilon + \epsilon/|A(s)| & \text{If } a = a^* \\ \epsilon/|A(s)| & \text{If } a \neq a^* \end{cases}$$

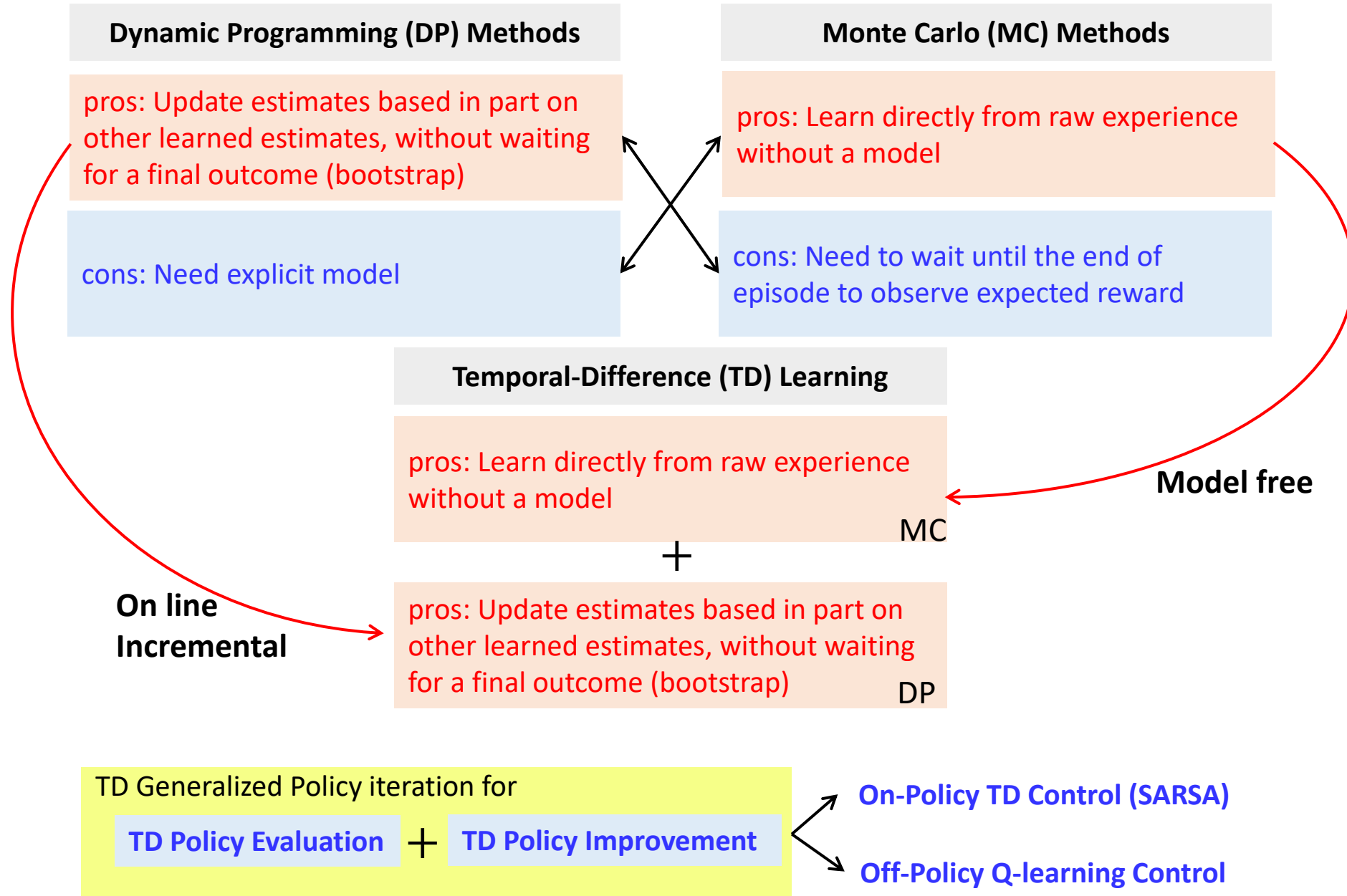
Policy improvement

Summary

- The Monte Carlo methods learn value functions and optimal policies from experience in the form of sample episodes
- Advantages are:
 - ✓ Can be used to learn optimal behavior directly from interaction with the environment with no models of environment dynamics
 - ✓ Can be used with simulation or sample models
 - ✓ It is efficient to focus Monte Carlo methods on a small subset of the states
 - ✓ Less harmed by violations of the Markov property. This is because they do not update their value estimates on the basis of the value estimates of successor states (do not use bootstrap)

3. Temporal Difference Learning

Introduction



Recall: Value Function

$$V^\pi(s) = \mathbb{E}_\pi(U_t | s_t = s)$$

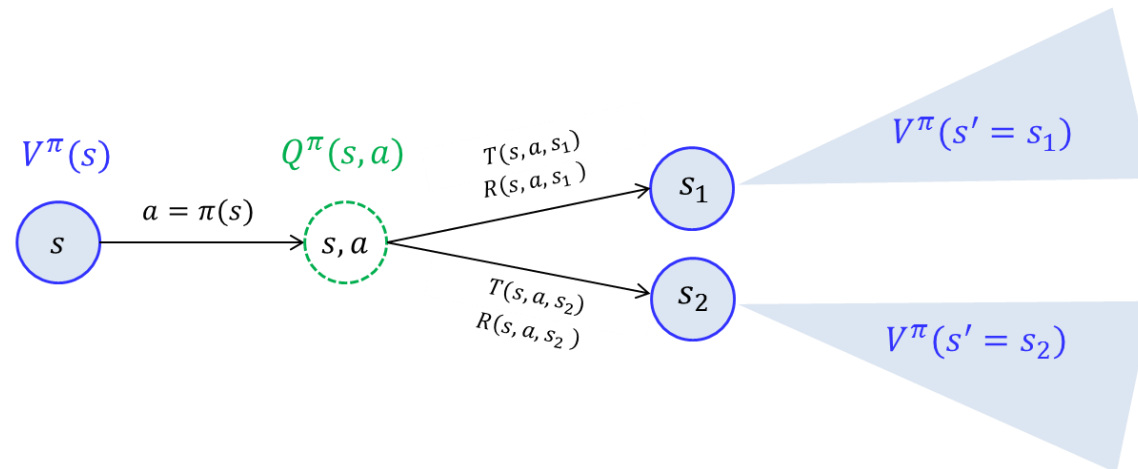
$$= \mathbb{E}_\pi(r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots | s_t = s) \text{ Complete episode}$$

$$= \mathbb{E}_\pi\left(\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s\right)$$

$$= \mathbb{E}_\pi\left(r_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k r_{t+k+2} \mid s_t = s\right)$$

$$= \mathbb{E}_\pi(r_{t+1} + \gamma V^\pi(s_{t+1}) | s_t = s)$$

Bootstrapping



Monte Carlo Policy Evaluation

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi(U_t | s_t = s) \\ &= \mathbb{E}_\pi\left(\sum_{k=0}^T \gamma^k r_{t+k+1} \mid s_t = s\right) \\ &= \mathbb{E}_\pi(r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots \mid s_t = s) \quad \text{A sampled episode} \end{aligned}$$

Constant- α MC :

After visiting s_t and receiving utility $u_t = r_{t+1} + r_{t+2} + r_{t+3} + \cdots + r_T$

$$V(s_t) \leftarrow V(s_t) + \alpha[u_t - V(s_t)]$$

$$V(s_t) \leftarrow V(s_t) + \alpha[\underbrace{r_{t+1} + r_{t+2} + r_{t+3} + \cdots + r_T}_{\text{Target}} - V(s_t)]$$

- The target of update is $u_t = r_{t+1} + r_{t+2} + r_{t+3} + \cdots + r_T$
- A sample reward u_t from a single episode is used for representing the expected reward. If the episode is long, u_t will be a lousy estimate (a single initialization)
- This is estimate because we use sampled value instead of expected utility

Temporal Difference Policy Evaluation

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi(U_t | s_t = s) \\ &= \mathbb{E}_\pi\left(\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s\right) \\ &= \mathbb{E}_\pi\left(r_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k r_{t+k+2} \mid s_t = s\right) \\ &= \mathbb{E}_\pi(r_{t+1} + \gamma V^\pi(s_{t+1}) | s_t = s) \end{aligned}$$

Bootstrapping

Temporal Difference Policy Evaluation ; $TD(0)$:

After visiting s_t and transiting to s_{t+1} with a single reward r_{t+1}

$$V(s_t) \leftarrow V(s_t) + \alpha [\underbrace{r_{t+1} + \gamma V(s_{t+1})}_{\text{Target}} - V(s_t)]$$

- Bootstrapping: the TD method updates the state value using the previous estimations
- The TD target is an estimate because
 - ✓ it uses the current estimate of $V(s_t)$,
 - ✓ it samples the expected value

$$\mathbb{E}_\pi(r_{t+1} + \gamma V^\pi(s_{t+1}) | s_t = s)$$

Temporal Difference Policy Evaluation

Algorithm : Tabular $TD(0)$ for estimating V^π

Initialize $V(s)$ arbitrarily, π to the policy to be evaluated

Repeat (for each episode):

Initialize s

Repeat (for **each step** of episode)

$a \leftarrow$ action given by π for s

Take action a ; observe reward r and next state s'

$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$

$s \leftarrow s'$

Until s is terminal

- Simple backups (MC method and TD methods) : Use a single sample success state

Recall:

- Full Backups (DP approach) : Use complete distribution of all possible successors

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') \{R(s, \pi(s), s') + \gamma V^\pi(s')\}$$

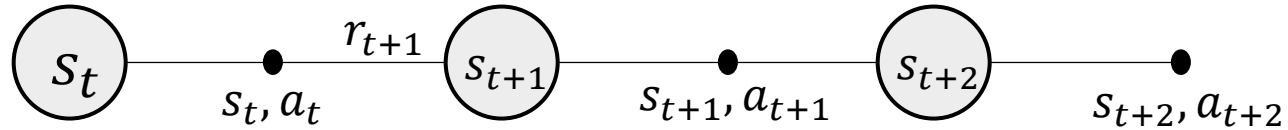
Advantages of TD Policy Evaluation (prediction)

What advantages do TD methods have over Monte Carlo and DP methods?

- TD methods learn their estimates on the basis of other estimates (Bootstrap)
- TD methods do not require a model of the environment, i.e., reward and state transition models
- TD methods can be naturally implemented in an **on-line**, fully incremental fashion:
 - ✓ Monte Carlo Method **must wait until the end of an episode**, because only then the return is revealed
 - ✓ TD methods operates with **a single transition of state and action** (a single time step) → advantages for continuous task and learning
- TD methods and Monte Carlo methods converge to V^π in the mean for a constant step-size if it is sufficiently small, and with probability 1 if the step-size parameter decreases
- In practice, TD methods have usually been found to converge faster than *constant* – α MC methods on stochastic tasks

Temporal Difference Policy Evaluation for Q function

As we estimate state value $V(s)$, we can estimate $Q(s, a)$ using a TD method



Temporal Difference Policy Evaluation for $Q(s, a)$ function

On each $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$ for a single episode:

Note that the action taken is given as data

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [\underbrace{r_{t+1} + \gamma Q(s_{t+1}, a_{t+1})}_{\text{Target}} - \underbrace{Q(s_t, a_t)}_{\text{Current estimate}}]$$

TD Generalized Policy iteration for

TD Policy Evaluation

+

TD Policy Improvement

- On-Policy TD Control (SARSA)
- Off-Policy TD Control (Q-learning)

Estimation and prediction problem

Decision making problems

Classification of RL

		How to estimate $V^*(s)$ and $Q^*(s, a)$	
		Monte Carlo method	Temporal Difference methods
How to explore ?		Non-Bootstrap	Bootstrap
	On-policy	On-policy Monte Carlo Control	SARSA
	Off-policy	Off-policy Monte Carlo Control	Q-Learning (SARSmxA)

- Episodic based
- Single-data-point based

SARSA: On-Policy TD Control

SARSA Algorithm

Initialize $Q(s, a)$ arbitrarily

Repeat (for each episode):

Initialize s

Choose a from s using policy derived from Q (e.g., $\epsilon - greedy$)

Repeat (for **each time step** of episode):

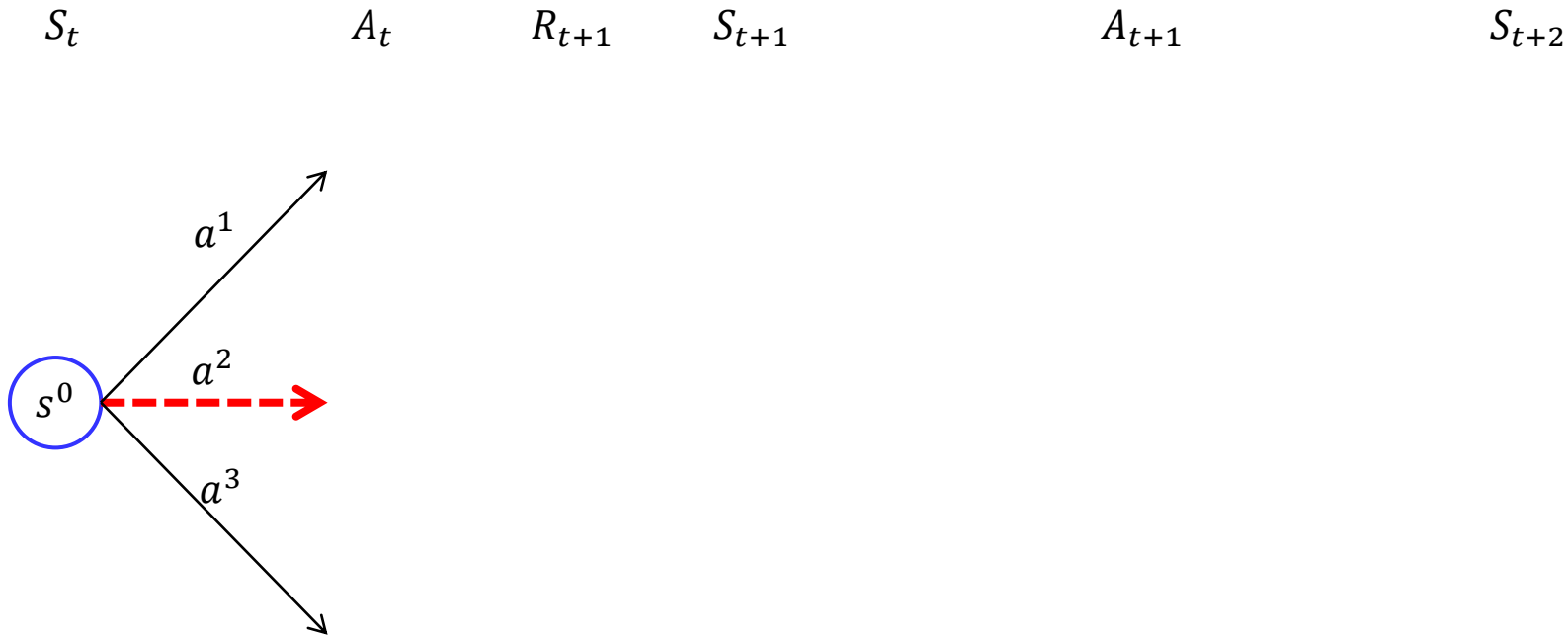
$\left\{ \begin{array}{l} \text{Take action } a \text{ given } s, \text{ observe } r, s' \\ \text{Choose } a' \text{ from } s' \text{ using policy derived from } Q \text{ (e.g., } \epsilon - greedy \text{)} \\ Q_{\pi}(s, a) \leftarrow Q_{\pi}(s, a) + \eta(r + \gamma Q_{\pi}(s', a') - Q_{\pi}(s, a)) \\ s \leftarrow s'; a \leftarrow a'; \end{array} \right.$

Behavioral policy
||
Estimation policy

Until s is terminal

- As in all on-policy methods, we continually estimate Q^{π} for the behavioral policy, and the same time change π toward greediness with respect to Q^{π}
- Converges with
 - ✓ All state-action pairs are visited an infinite number of times
 - ✓ The policy converges in the limit to the greedy policy (i.e., $\epsilon - greedy$ with $\epsilon = 1/t$)

SARSA: On-Policy TD Control

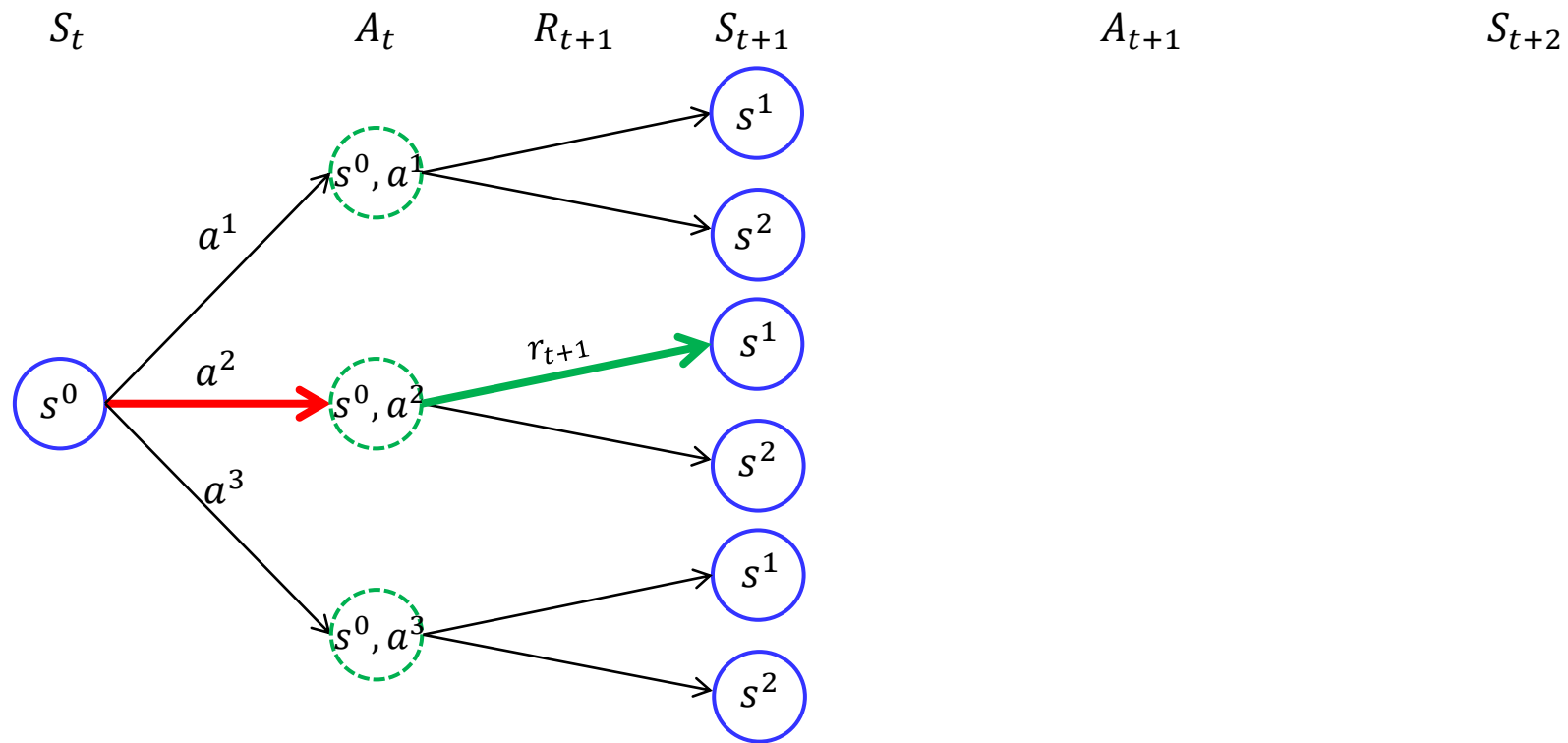


Choose a_t from $s_t = s^0$ using current Q

$$a_t = \begin{cases} \operatorname{argmax}_a Q(s_t = s^0, a) & \text{with prob } 1 - \epsilon \\ \text{random action} & \text{with prob } \epsilon \end{cases}$$

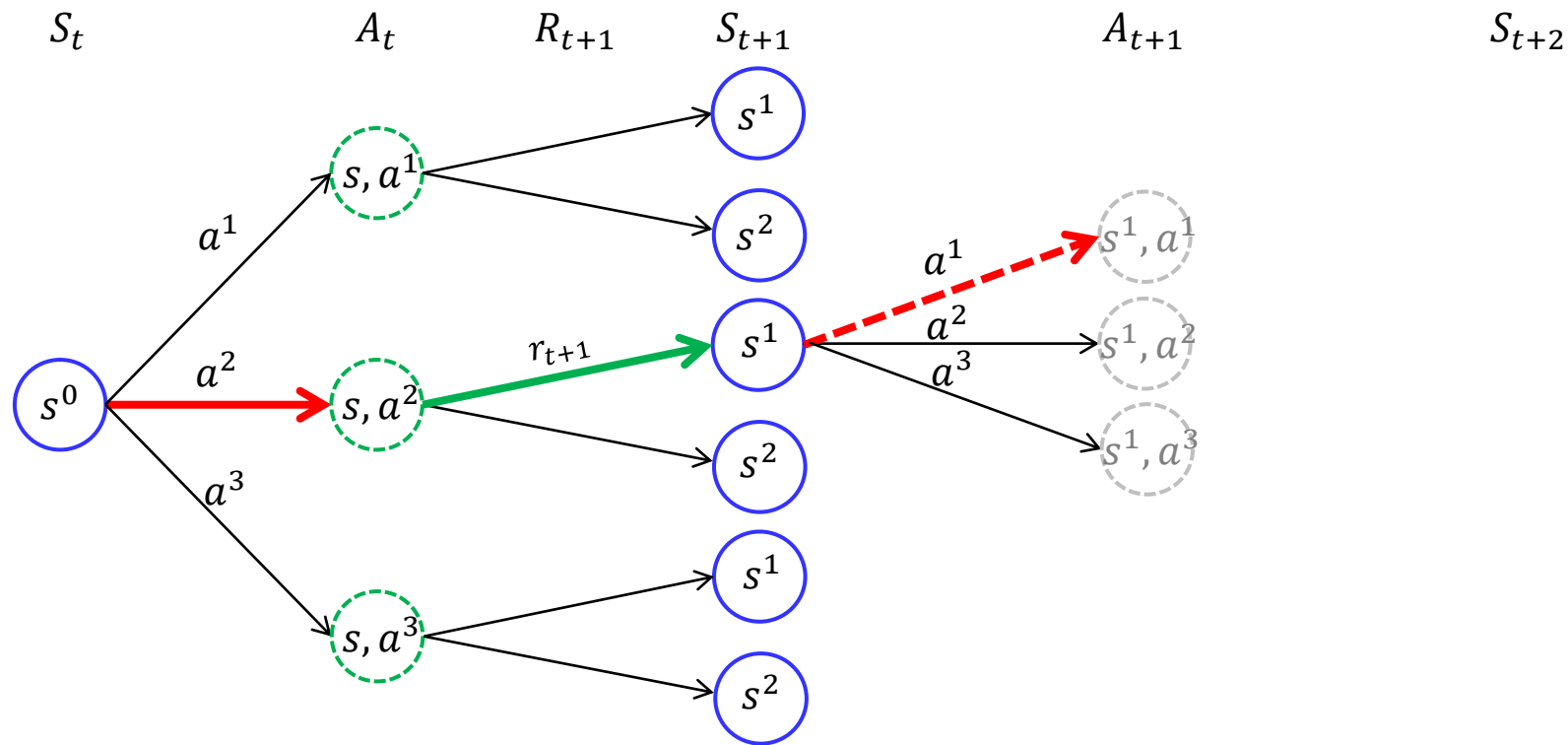
Assume a^2 is chosen

SARSA: On-Policy TD Control



Take action $a_t = a^2$ given $s_t = s^0$ and observe r_{t+1} and $s_{t+1} = s^1$

SARSA: On-Policy TD Control

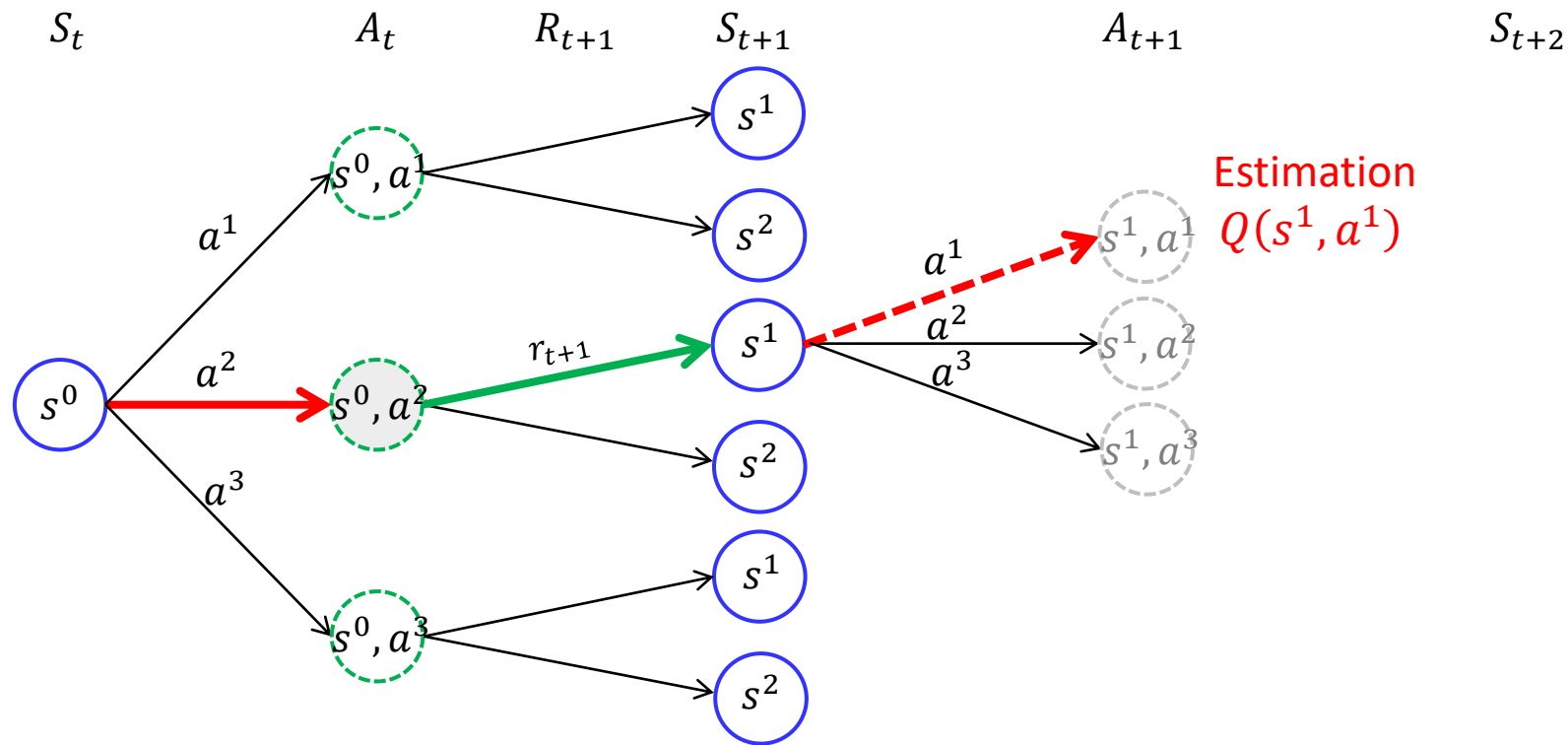


Choose a_{t+1} from $s_{t+1} = s^1$ using current Q

$$a_{t+1} = \begin{cases} \operatorname{argmax}_a Q(s_{t+1} = s^1, a) & \text{with prob } 1 - \epsilon \\ \text{random action} & \text{with prob } \epsilon \end{cases}$$

Assume a^1 is chosen

SARSA: On-Policy TD Control

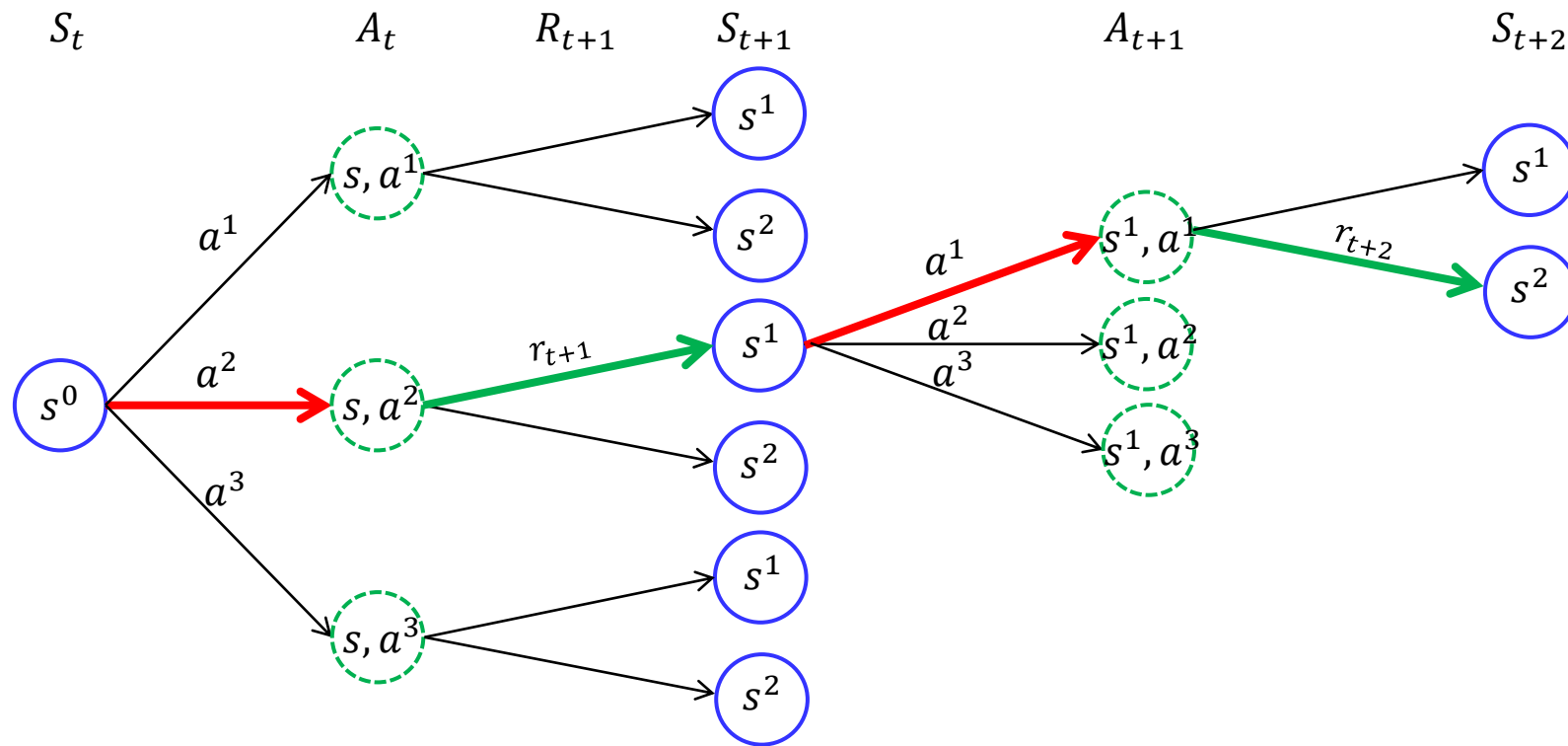


Update Q function with the **estimation** $Q(s_{t+1}, a_{t+1})$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

$$\rightarrow Q(s^0, a^2) \leftarrow Q(s^0, a^2) + \alpha[r_{t+1} + \gamma Q(s^1, a^1) - Q(s^0, a^2)]$$

SARSA: On-Policy TD Control



Take action $a_{t+1} = a^1$ given $s_{t+1} = s^1$ and observe r_{t+2} and $s_{t+2} = s^2$

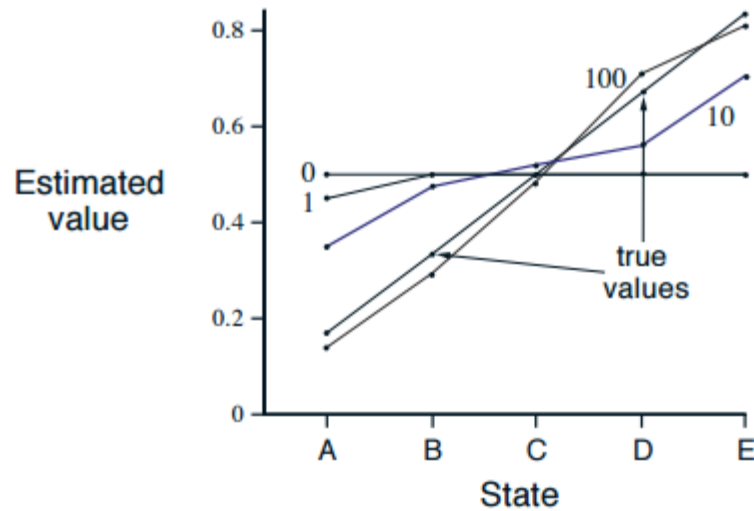
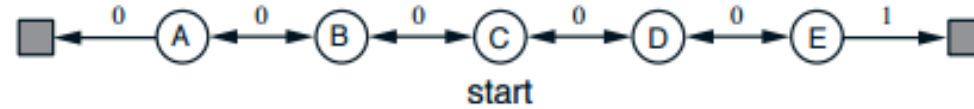
Why Q-learning is considered as Off-Policy method

The diagram illustrates the off-policy nature of Q-learning. A green arrow points from the Q-value update equation to the state s in the tuple (s, a, r, s', a') . Blue curved arrows point from $\pi(s)$ to s and from $\pi(s')$ to s' . Red straight arrows point from a to s and from a' to s' .

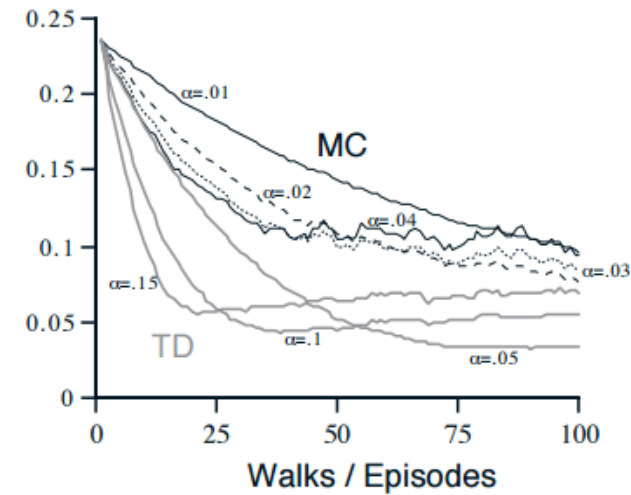
$$Q(s, a) \leftarrow Q(s, a) + \eta(r + \gamma Q(s', a') - Q_{\pi}(s, a))$$

SARSA: On-Policy TD Control : Windy Grid world Example

A small Markov process for generating random walk

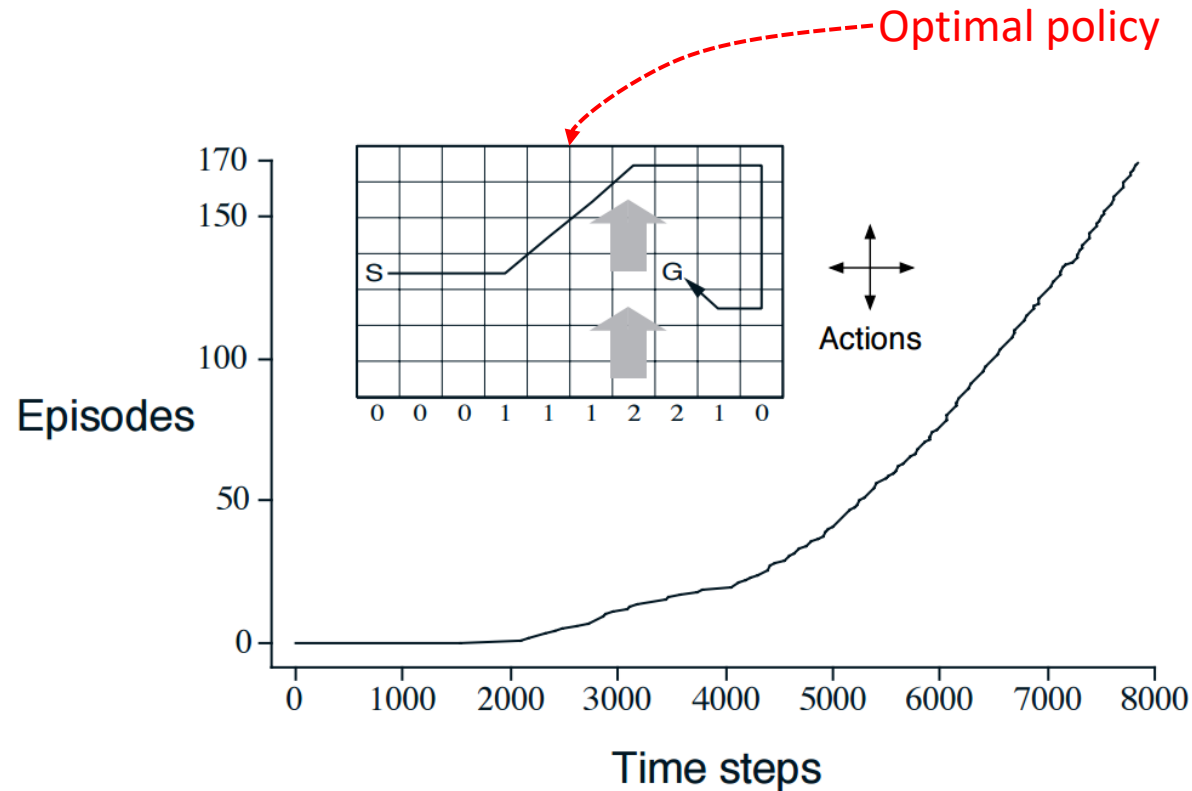
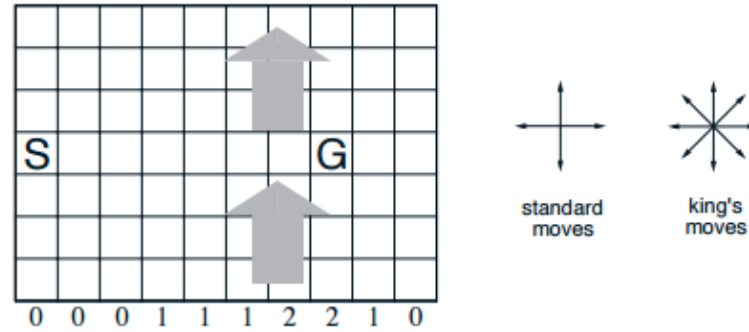


RMS error,
averaged
over states



SARSA: On-Policy TD Control : Windy Grid world Example

State transition is encoded by this figure



Classification of RL

		How to estimate $V^*(s)$ and $Q^*(s, a)$	
		Monte Carlo method	Temporal Difference methods
How to explore ?		Non-Bootstrap	Bootstrap
	On-policy	On-policy Monte Carlo Control	SARSA
	Off-policy	Off-policy Monte Carlo Control	Q-Learning (SARSmAxA)

- Episodic based
- Single-data-point based

Q-Learning: Off-Policy TD Control

On-Policy TD Control (SARSA)

Choose a' from s' using policy derived from Q (e.g., $\epsilon - greedy$)

$$Q(s, a) \leftarrow Q(s, a) + \eta(r + \gamma Q(s', a') - Q(s, a))$$


Off-Policy TD Control (Q-learning)

$$Q(s, a) \leftarrow Q(s, a) + \eta(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

- The **max over a** rather than **taking the a based on the current policy** is the principle difference between Q-learning and SARSA.
- The learned action-value function Q directly approximates Q^* independent of the policy being followed
- Converges with
 - ✓ All state-action pairs are visited an infinite number of times
 - ✓ The policy converges in the limit to the greedy policy (i.e., $\epsilon - greedy$ with $\epsilon = 1/t$)

Q-Learning: Off-Policy TD Control

Q learning

Initialize $Q(s, a)$ arbitrarily

Repeat (for each episode):

Initialize s

Repeat (for each time step of episode):

Choose a from s using policy derived from Q (e.g., $\epsilon - greedy$) **Behavioral policy**

Take action a , observe r, s'

$$Q(s, a) \leftarrow Q(s, a) + \eta \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

$$s \leftarrow s'$$

Until s is terminal

Estimation policy

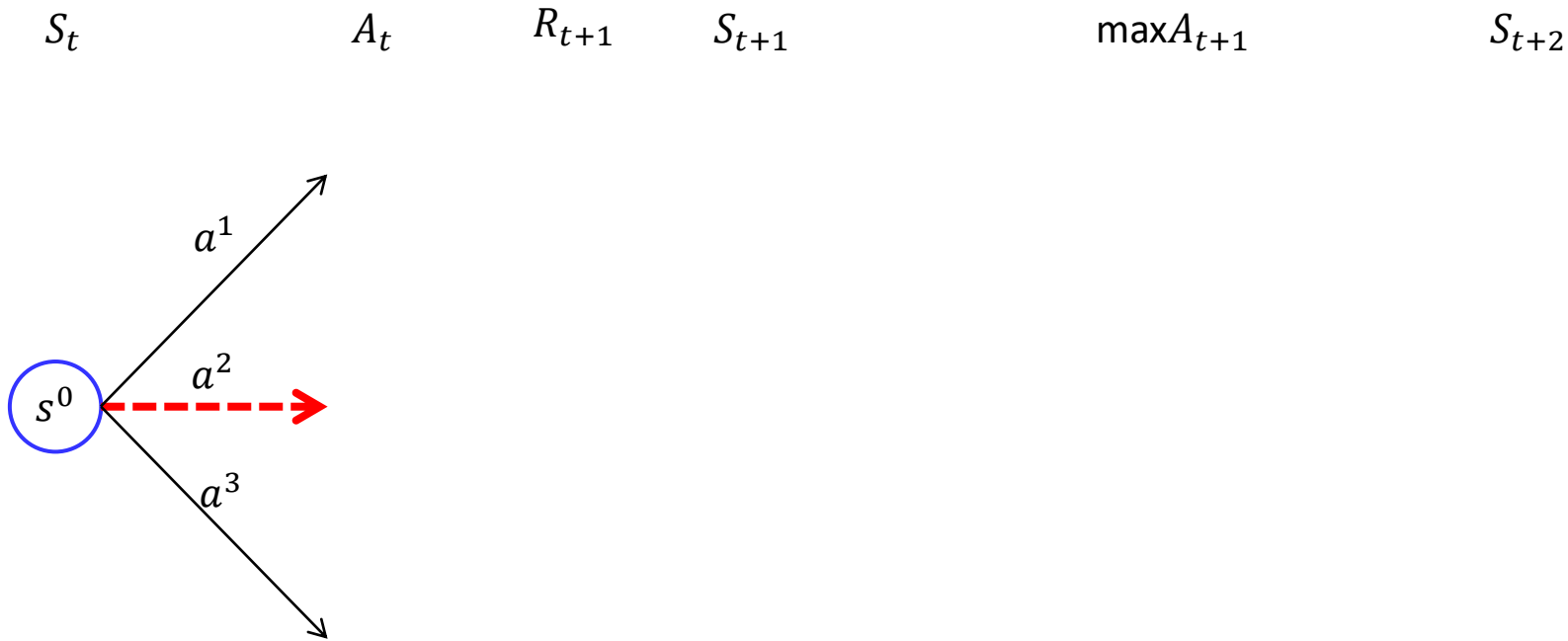
(Always try to estimate the optimal policy)

-Estimation can be greedy)

$a^* = \operatorname{argmax}_{a'} Q(s', a)$ is **not** used in the next state!!!

At the next state s' , Choose a using policy derived from Q (e.g., $\epsilon - greedy$)

Q-Learning: Off-Policy TD Control

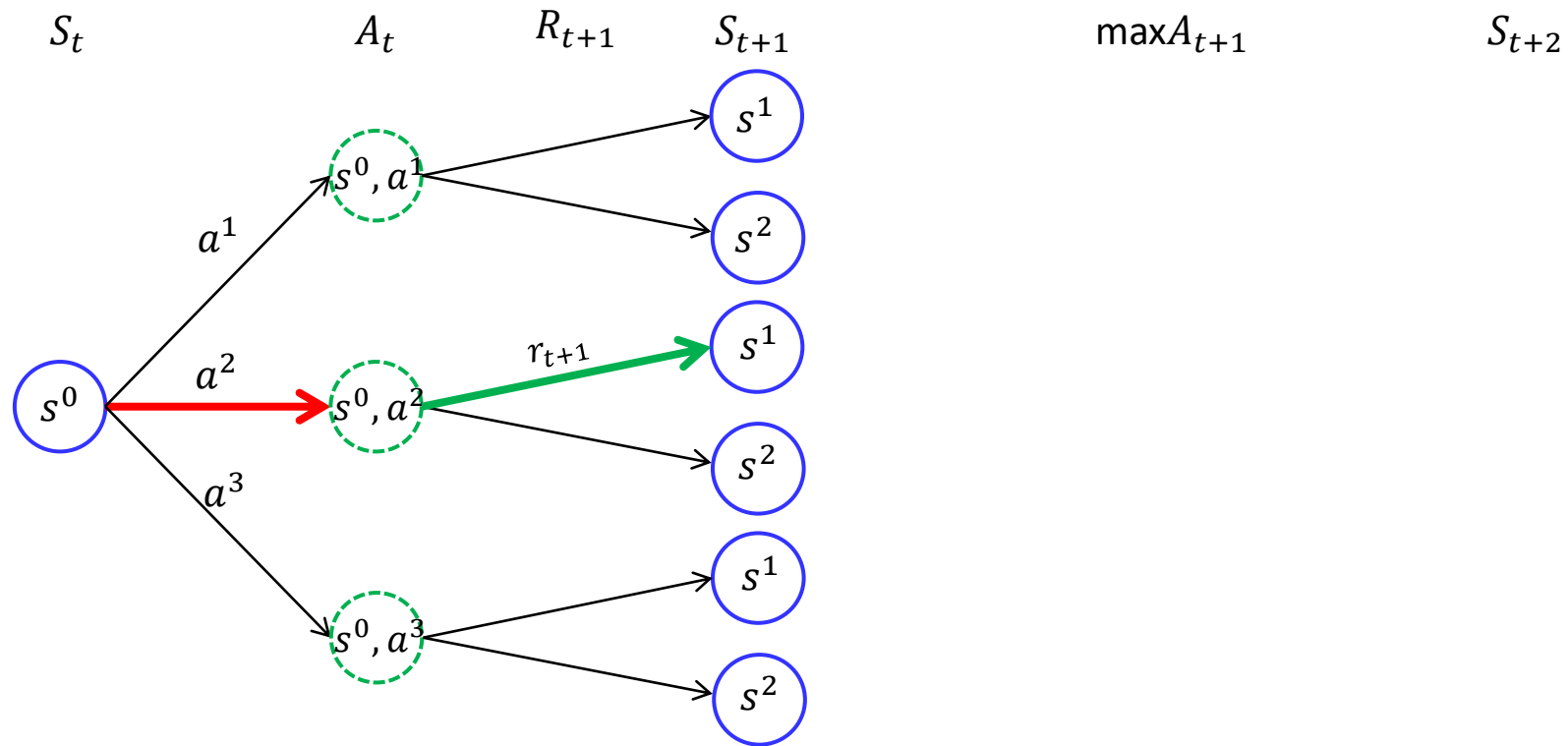


Choose a_t from $s_t = s^0$ using current Q

$$a_t = \begin{cases} \operatorname{argmax}_a Q(s_t = s^0, a) & \text{with prob } 1 - \epsilon \\ \text{random action} & \text{with prob } \epsilon \end{cases}$$

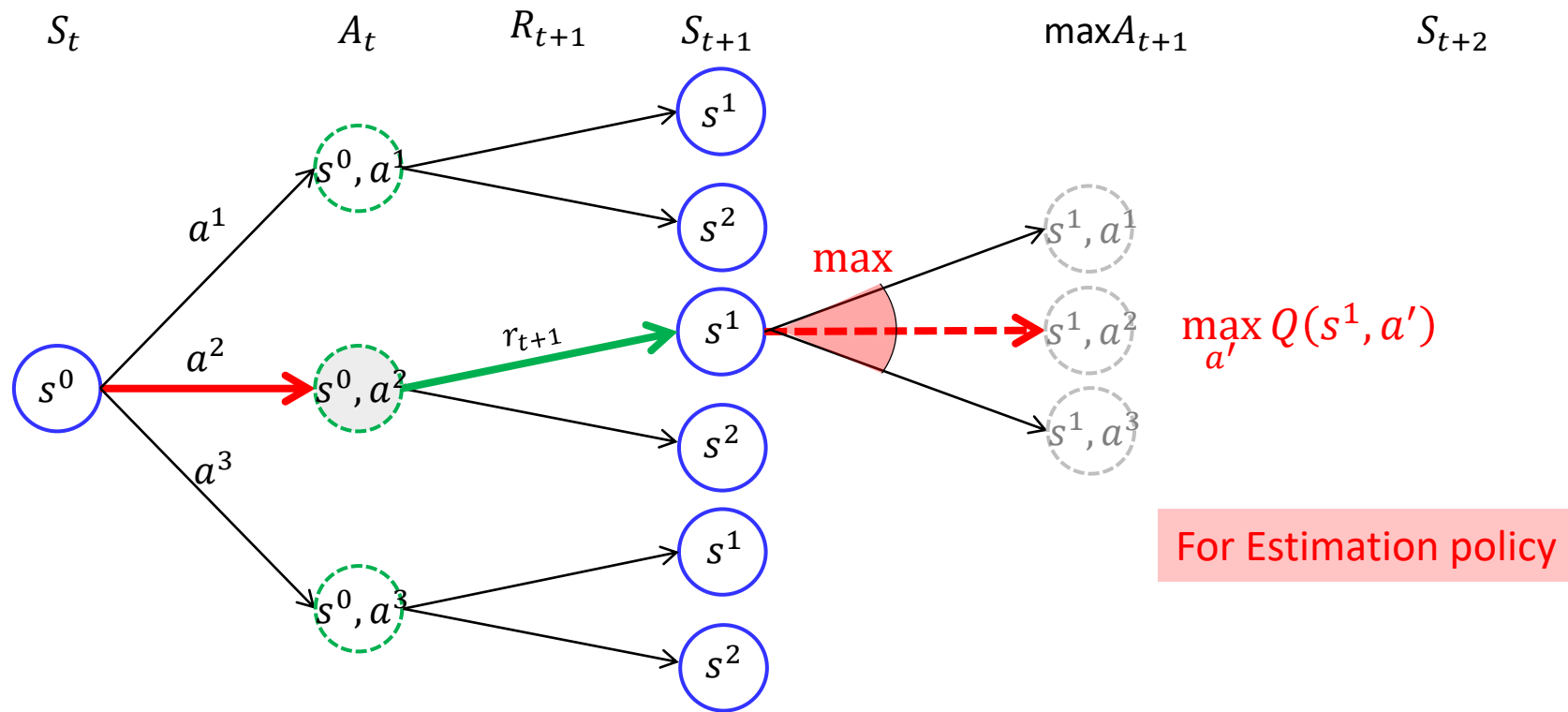
Assume a^2 is chosen

Q-Learning: Off-Policy TD Control



Take action $a_t = a^2$ given $s_t = s^0$ and observe r_{t+1} and $s_{t+1} = s^1$

Q-Learning: Off-Policy TD Control

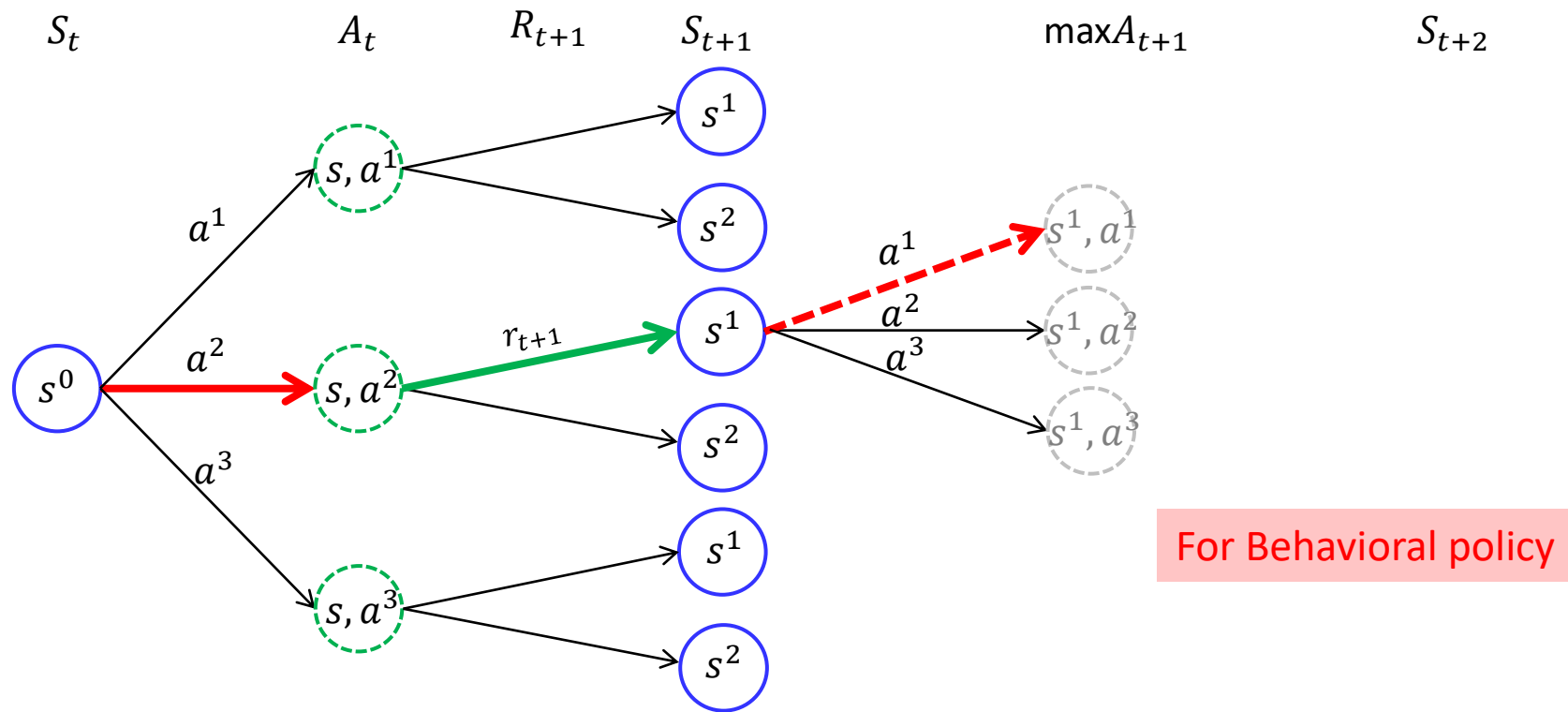


Update Q function with the $\max_{a'} Q(s^1, a')$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s, a') - Q(s_t, a_t) \right]$$

$$\rightarrow Q(s^0, a^2) \leftarrow Q(s^0, a^2) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s^1, a') - Q(s^0, a^2) \right]$$

Q-Learning: Off-Policy TD Control

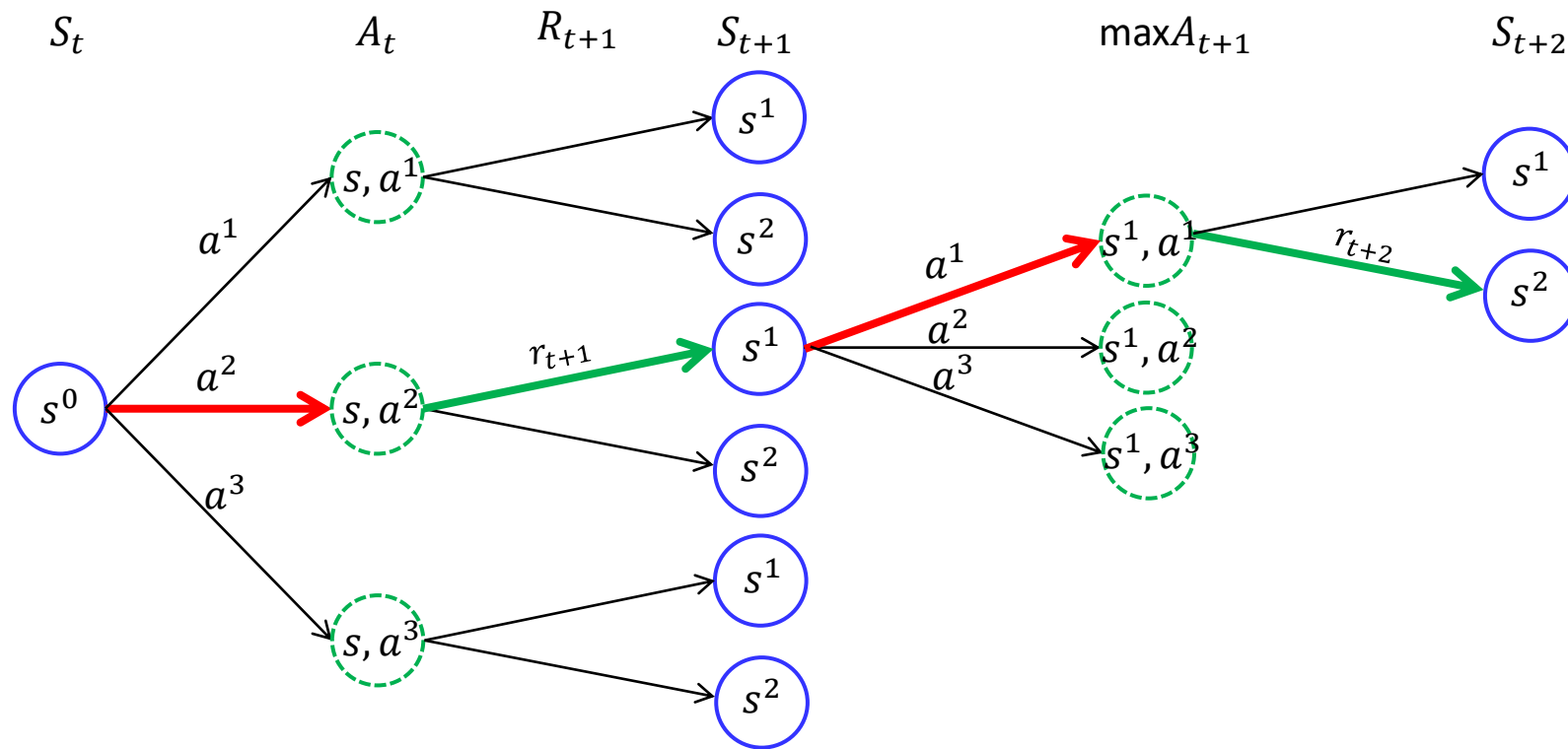


Choose a_{t+1} from $s_{t+1} = s^1$ using current Q

$$a_{t+1} = \begin{cases} \operatorname{argmax}_a Q(s_{t+1} = s^1, a) & \text{with prob } 1 - \epsilon \\ \text{random action} & \text{with prob } \epsilon \end{cases}$$

Assume a^1 is chosen

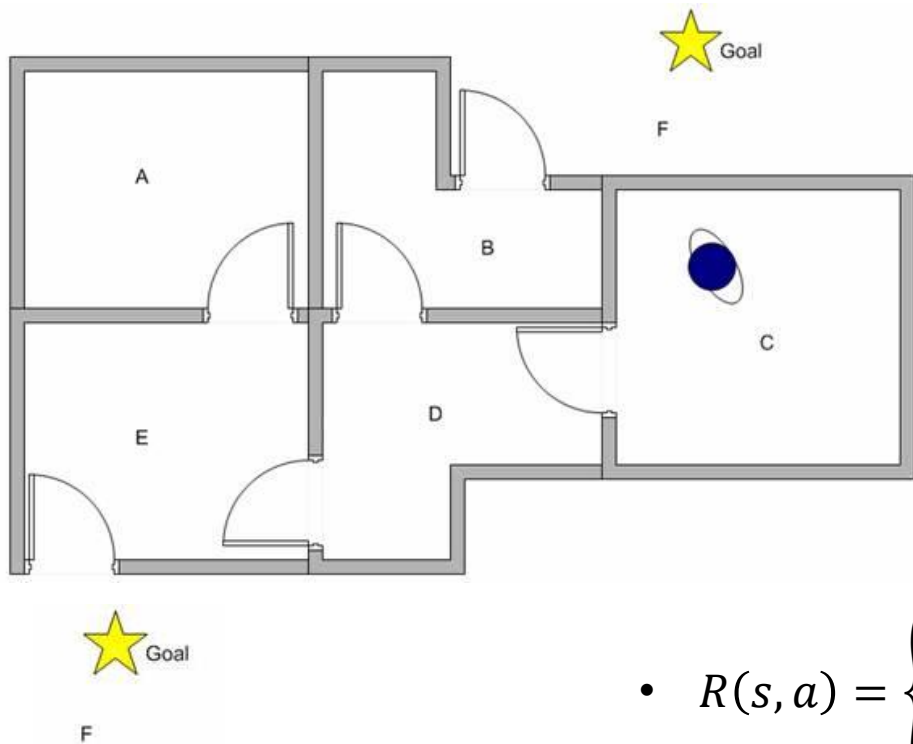
Q-Learning: Off-Policy TD Control



Take action $a_{t+1} = a^1$ given $s_{t+1} = s^1$ and observe r_{t+2} and $s_{t+2} = s^2$

Q-learning Step-by-Step Example

- The agent can pass one room to another but has no knowledge of the building
- That is, it does not know which sequence of doors the agent must pass to go outside the building
- Assume the agent is now in room C, and would like to reach outside the building (state F)

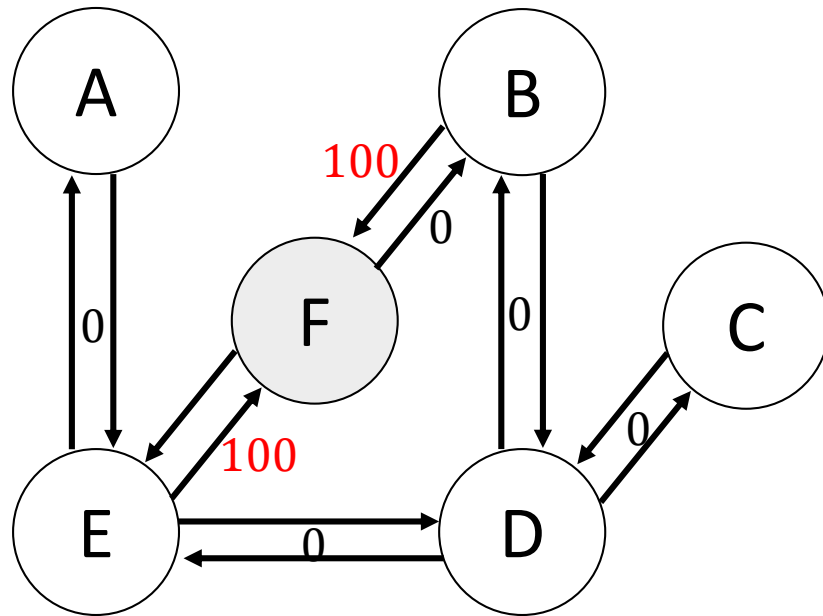


$$MDP = \{\mathcal{S}, \mathcal{A}, T, R, \gamma\}$$

- $s \in \mathcal{S} = \{A, B, C, D, E, F\}$
- $a \in \mathcal{A} = \{A, B, C, D, E, F\}$
e.g., $\mathcal{A}(s = D) = \{B, C, E\}$
- $T(s, a) = \begin{cases} 1, & \text{if move is allowed} \\ 0, & \text{if move is not allowed} \end{cases}$
e.g., $T(C, D) = 1$

- $R(s, a) = \begin{cases} 0 & \text{if move to } a \text{ is allowed and } a \neq F \\ 100 & \text{if move to } a \text{ is allowed and } a = F \end{cases}$
- $\gamma = 0.8$

Q-learning Step-by-Step Example



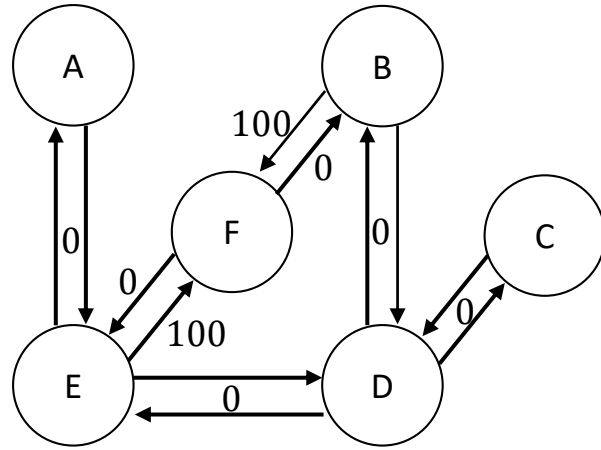
$R(s, a) =$

$s \backslash a$	A	B	C	D	E	F
A	-	-	-	-	0	-
B	-	-	-	0	-	100
C	-	-	-	0	-	-
D	-	0	0	-	0	-
E	0	-	-	0	-	100
F	-	0	-	-	0	100

Q Learning update rule:

$$Q(s, a) \leftarrow Q(s, a) + \eta \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

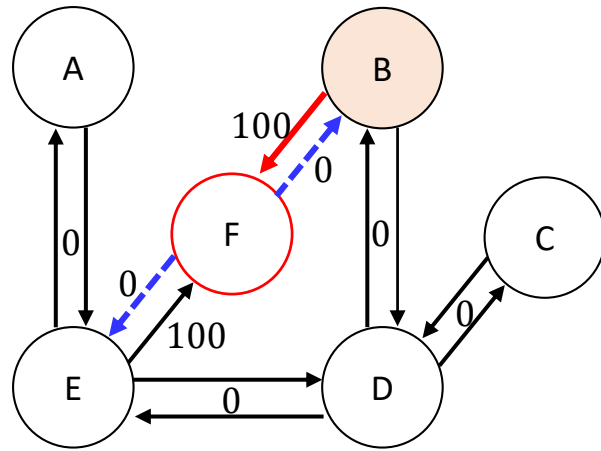
Q-learning Step-by-Step Example

$$Q \text{ Learning update rule: } Q(s, a) \leftarrow Q(s, a) + \eta \left(r + \gamma \max_a Q(s', a) - Q(s, a) \right)$$


[illegible]

Q-learning Step-by-Step Example

Q Learning update rule: $Q(s, a) \leftarrow Q(s, a) + \eta \left(r + \gamma \max_a Q(s', a) - Q(s, a) \right)$



$$Q(s, a) = \begin{matrix} & \begin{matrix} A & B & C & D & E & F \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \\ F \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 50 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

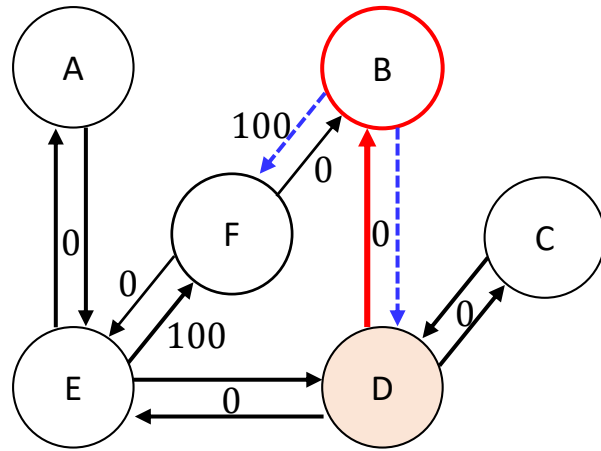
1. Assume the initial state is B and take action F randomly (stochastic policy):

$$Q(B, F) \leftarrow Q(B, F) + 0.5 \left(R(B, F) + 0.8 \max_a \{Q(F, B), Q(F, E)\} - Q(B, F) \right)$$
$$Q(B, F) \leftarrow 0 + 0.5(100 + 0.8 \times 0 - 0) = 50$$

2. Because the state F is the final state, the episode is over

Q-learning Step-by-Step Example

Q Learning update rule: $Q(s, a) \leftarrow Q(s, a) + \eta \left(r + \gamma \max_a Q(s', a) - Q(s, a) \right)$


$$Q(s, a) =$$

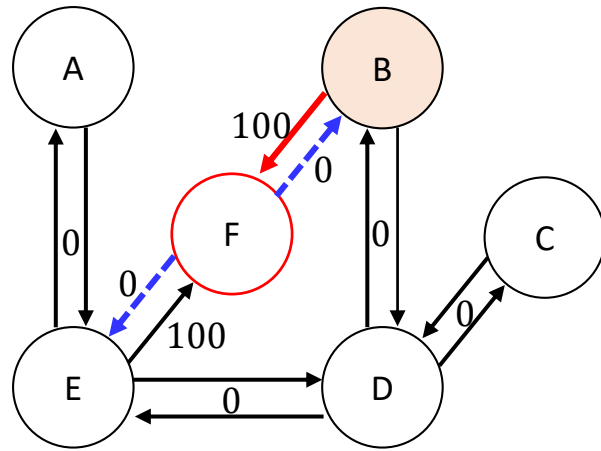
	A	B	C	D	E	F
A	0	0	0	0	0	0
B	0	0	0	0	0	50
C	0	0	0	0	0	0
D	0	20	0	0	0	0
E	0	0	0	0	0	0
F	0	0	0	0	0	0

1. Assume the initial state is D and take action B randomly (stochastic policy):

$$Q(D, B) \leftarrow Q(D, B) + 0.5 \left(R(D, B) + 0.8 \max_a \{ Q(B, F)^{50}, Q(B, D)^0 \} - Q(D, B) \right)$$
$$Q(D, B) \leftarrow 0 + 0.5(0 + 0.8 \times 50 - 0) = 20$$

Q-learning Step-by-Step Example

Q Learning update rule: $Q(s, a) \leftarrow Q(s, a) + \eta \left(r + \gamma \max_a Q(s', a) - Q(s, a) \right)$



$$Q(s, a) = \begin{matrix} & \begin{matrix} A & B & C & D & E & F \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \\ F \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 75 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 20 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

1. Assume the initial state is D and take action B randomly (stochastic policy):

$$Q(D, B) \leftarrow Q(D, B) + 0.5 \left(R(D, B) + 0.8 \max_a \{Q(B, F), Q(B, D)\} - Q(D, B) \right)$$
$$Q(D, B) \leftarrow 0 + 0.5(0 + 0.8 \times 50 - 0) = 20$$

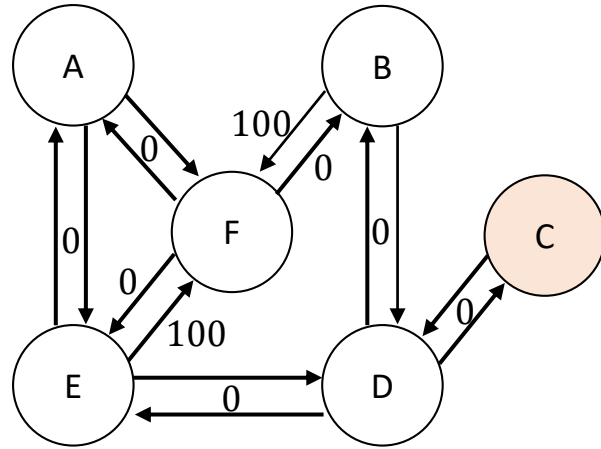
2. The next state is B and take an action of F randomly):

$$Q(B, F) \leftarrow Q(B, F) + 0.5 \left(R(B, F) + 0.8 \max_a \{Q(F, B), Q(F, E)\} - Q(B, F) \right)$$
$$Q(B, F) \leftarrow 50 + 0.5(100 + 0.8 \times 0 - 50) = 75$$

3. Because the state F is the final state, the episode is over

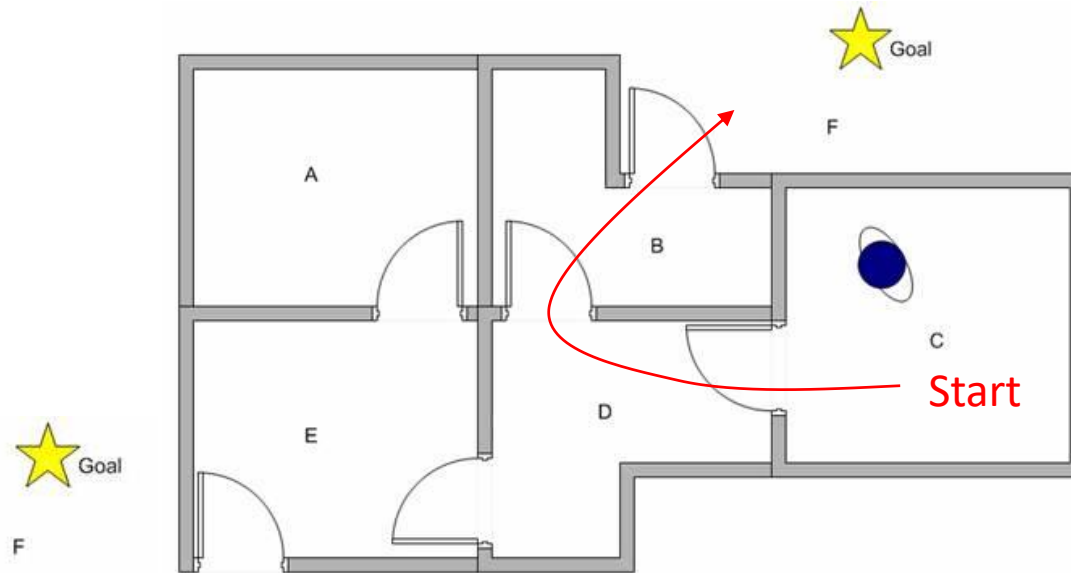
Q-learning Step-by-Step Example

Q Learning update rule: $Q(s, a) \leftarrow Q(s, a) + \eta \left(r + \gamma \max_a Q(s', a) - Q(s, a) \right)$

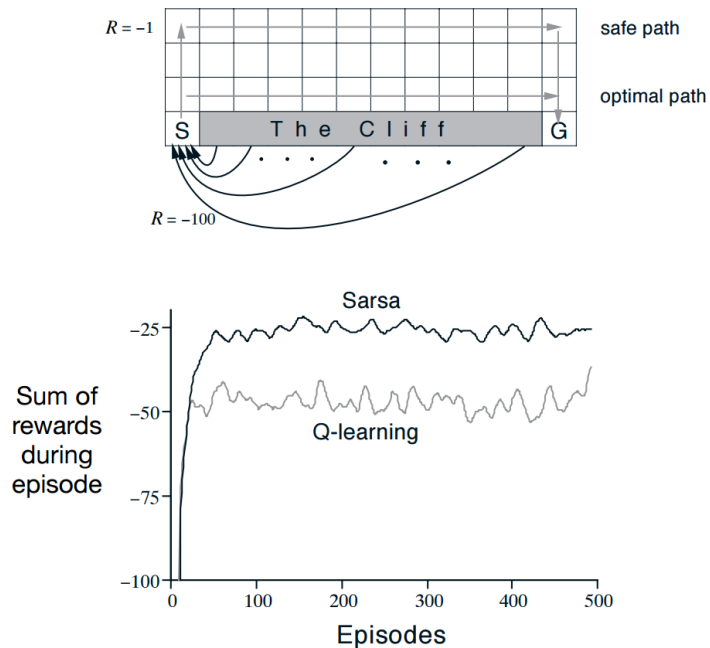


$$Q^*(s, a) = \begin{matrix} & \begin{matrix} A & B & C & D & E & F \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \\ F \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 400 & 0 \\ 0 & 0 & 0 & 320 & 0 & 500 \\ 0 & 0 & 0 & 320 & 0 & 0 \\ 0 & 400 & 256 & 0 & 400 & 0 \\ 320 & 0 & 0 & 320 & 0 & 500 \\ 0 & 400 & 0 & 0 & 400 & 500 \end{bmatrix} \end{matrix}$$

After convergence



Cliff Walking



The path away from the cliff

- Take longer
- A wrong action will not hurt you as much

walk near the cliff

- Faster
- a wrong action deterministically causes falling off the cliff.

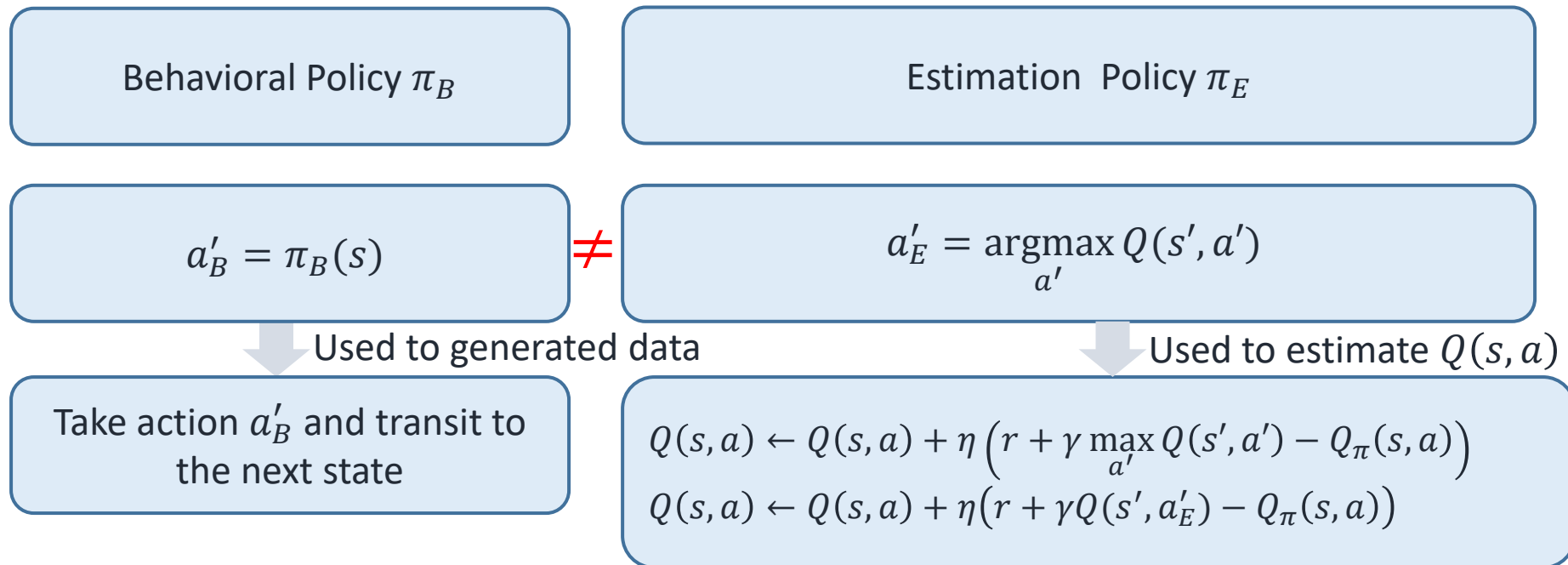
- **SARSA** learns about a policy that sometimes takes optimal actions (as estimated) and sometimes explores other actions (Estimation policy = Behavioral policy)
 - SARSA will learn to be careful in an environment where exploration is costly
- **Q-learning** learns about the policy that doesn't explore and only takes optimal (as estimated) actions
 - The optimal policy does not capture the risk of exploratory action

The cliff example shows why such a non-optimal policy could be sometimes very useful

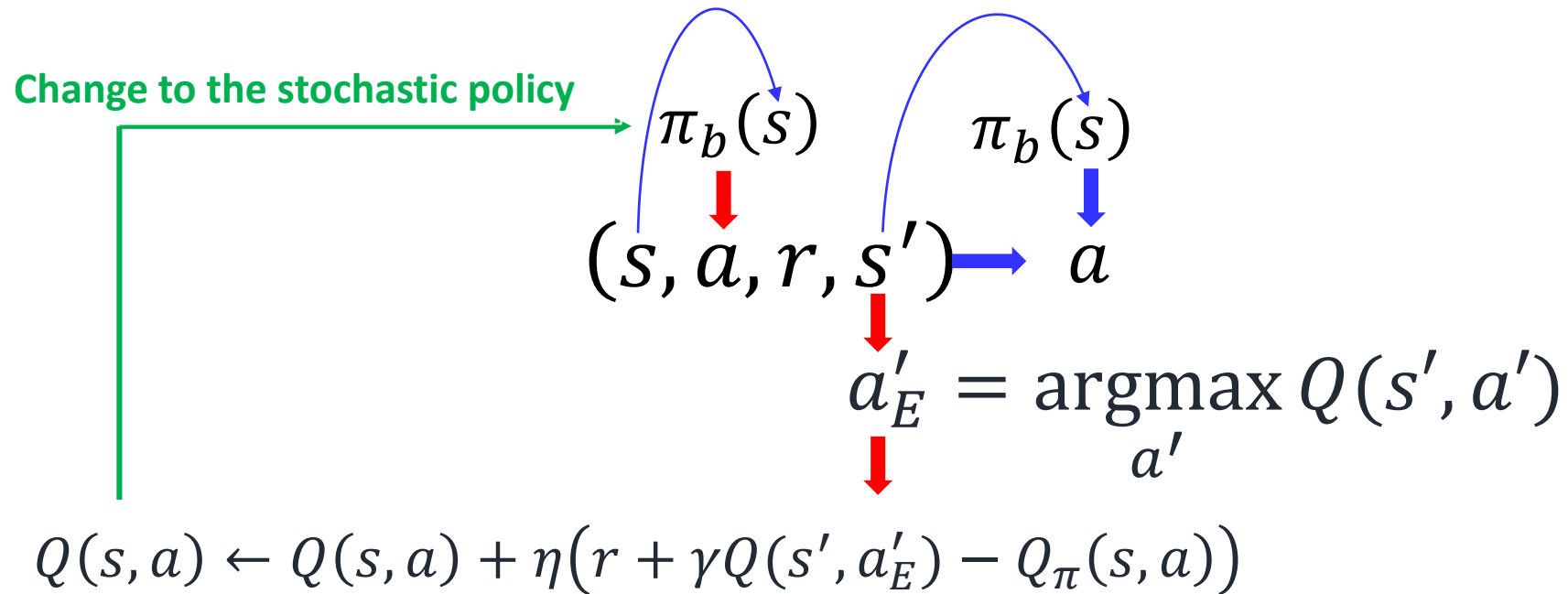
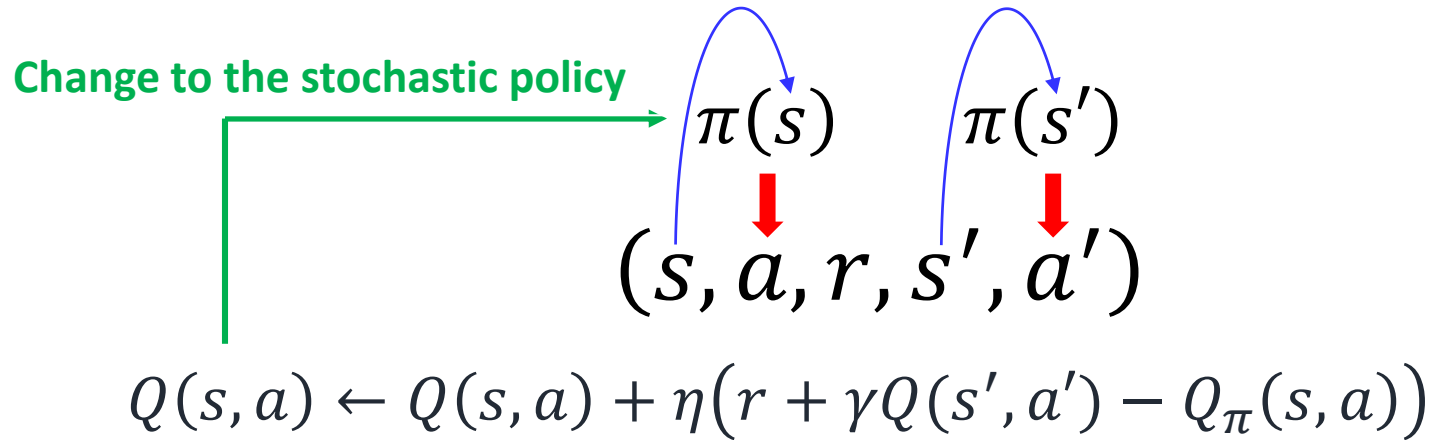
Why Q-learning is considered as Off-Policy method

- Q-learning updates are done regardless to the actual action chosen for next state (behavioral policy)
- That is, for estimation, it just assumes that we are always choosing the argmax one

$$a_{t+1} = \operatorname{argmax}_a Q(s_{t+1} = s^1, a)$$



Why Q-learning is considered as Off-Policy method



Greedy policy (deterministic)

Why Q-learning is considered as Off-Policy method

Consider the extreme case:

Suppose you were to take a completely random action on each step (if epsilon greedy exploration is used, set epsilon to 1).

- SARSA is literally learning the value of the random policy while acting randomly
- Q-learning is learning the value of the optimal policy, but is **acting** randomly.

Q-learning with function approximation

Q-learning (stochastic gradient update)

$$Q(s, a) \leftarrow Q(s, a) + \eta \left[\left(r + \gamma \max_a Q(s', a) \right) - Q(s, a) \right]$$

Problem:

- Basic Q-Learning keeps a table of all q-values
- We cannot possibly learn about every single state!
 - Too many states to visit them all in training
 - Too many states to hold the q-tables in memory
- Doesn't generalize to unseen states/actions

Solution:

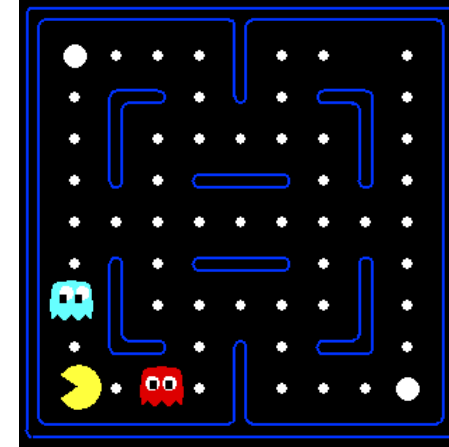
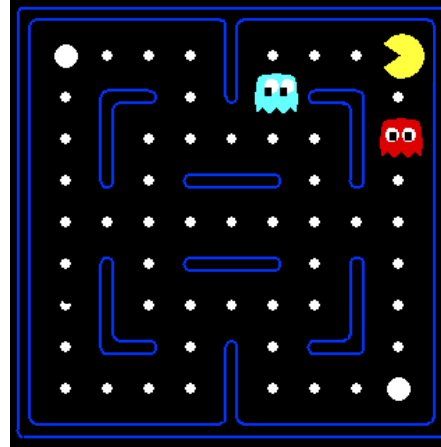
- Learn about some small number of training states from experience
- Generalize that experience to new, similar situations
 - fundamental idea in machine learning

Q-learning with function approximation

Obtained experience:
This state is bad!



Complete new states



- Are these states are good or bad?
- For table representation of $Q(s, a)$, **we cannot answer**
- **We can represent $Q(s, a)$ using the properties of the current state s :**

$$\begin{aligned} Q(s, a; w) &= w_1 \phi_1(s, a) + w_2 \phi_2(s, a) + \dots + w_n \phi_n(s, a) \\ &= w^T \phi(s, a) \end{aligned}$$

$\phi_i(s, a)$: distance to closet ghost, number of ghost, distance to foods

Q-learning with function approximation

Approximate $Q(s, a)$ as a function:

$$\begin{aligned} Q(s, a; w) &= w_1 \phi_1(s, a) + w_2 \phi_2(s, a) + \dots + w_n \phi_n(s, a) \\ &= w^T \phi(s, a) \end{aligned}$$

w : weight vector $\phi(s, a)$: features vector

- Features, $\phi(s, a) = \{\phi_1(s, a), \dots, \phi_n(s, a)\}$, are supposed to be properties of the state-action (s, a) pair that are indicative of the quality of taking action a and state s

Q-learning with function approximation

Algorithm : Q-learning with function approximation

On each (s, a, r, s') :

$$w \leftarrow w + \eta \left[\underbrace{\left(r + \gamma \max_a \hat{Q}(s', a; w) \right)}_{\text{Target}} - \underbrace{\hat{Q}(s, a; w)}_{\text{Estimate}} \right] \phi(s, a)$$

This is equivalent to find the weight w that maximizes the following objective function

$$\min_w \frac{1}{2} \sum_{(s, a, r, s')} \left(\underbrace{\left(r + \gamma \max_a \hat{Q}(s', a; w) \right)}_{\text{Target}} - \underbrace{\hat{Q}(s, a; w)}_{\text{Estimate}} \right)^2$$

For a single transition (s, a, r, s') , the error can be expressed as:

$$\begin{aligned} \text{Error}(w) &= \frac{1}{2} \left(\left(r + \gamma \max_a \hat{Q}(s', a; w) \right) - \hat{Q}(s, a; w) \right)^2 \\ &= \frac{1}{2} \left(\left(r + \gamma \max_a \hat{Q}(s', a; w) \right) - \sum_{k=1}^n w_k \phi_k(s, a) \right)^2 \quad \because \hat{Q}(s, a; w) = \sum_{k=1}^n w_k \phi_k(s, a) \\ \frac{\partial \text{Error}(w)}{\partial w_k} &= - \left(\left(r + \gamma \max_a \hat{Q}(s', a; w) \right) - \sum_{k=1}^n w_k \phi_k(s, a) \right) \phi_k(s, a) \end{aligned}$$

$$w_k \leftarrow w_k + \eta \left[\left(r + \gamma \max_a \hat{Q}(s', a; w) \right) - \hat{Q}(s, a; w) \right] \phi_k(s, a)$$

Deep Reinforcement Learning

Use a neural network for estimating $Q(s, a)$

Playing Atari [Google DeepMind, 2013]:

$a^* = \pi(s =$
move left
move right
stroke
 $)$



Figure 1: Atari Breakout game. Image credit: DeepMind.

- last 4 frames (images) $\Rightarrow$ 3-layer NN $\Rightarrow$ keystroke (move left, right, stroke)
- ϵ -greedy, train over 10M frames with 1M replay memory
- Human-level performance on some games (breakout)

Q-Learning with Neural Network

Screen size : 84×84 and convert to grayscale with 256 gray levels

State size = $256^{84 \times 84 \times 4} \approx 10^{67970}$ more than the number of atoms in the known universe

$$Q(s, a) = \left. \begin{array}{c} \text{ } \\ \text{ } \\ \text{ } \end{array} \right\} 10^{67970}$$

- Need to represent this large Q table using a function approximation (Deep learning)
- Need to estimate Q-values for states that have never been seen before (generalization)

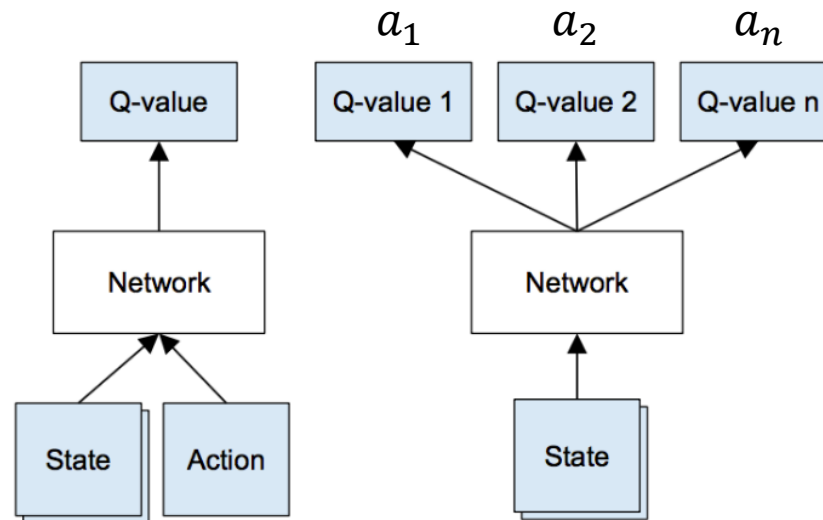


Figure 3: Left: Naive formulation of deep Q-network. Right: More optimized architecture of deep Q-network, used in DeepMind paper.

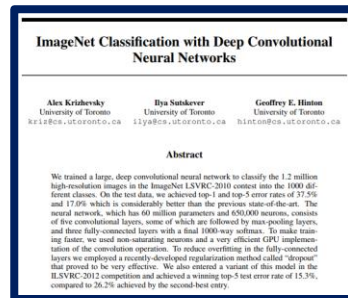
Off-Policy TD Control (Q-learning)

- Based on a single transition, i.e., state-action pair
- Online setting: Learn and take action continuously
- Exploration and Exploitation : Nee to learn and optimize at the same time
- Monte Carlo vs. Bootstrapping

4. Deep Q-Learning

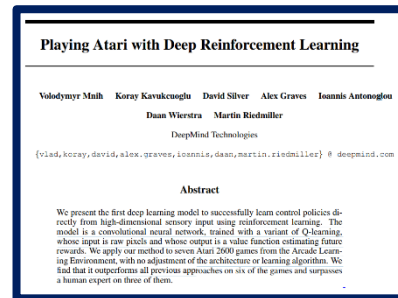
Deep Q-Network

Alexnet (NIPS 2012)



Dec 3, 2012

DQN (Arxiv version)



Dec 19, 2013

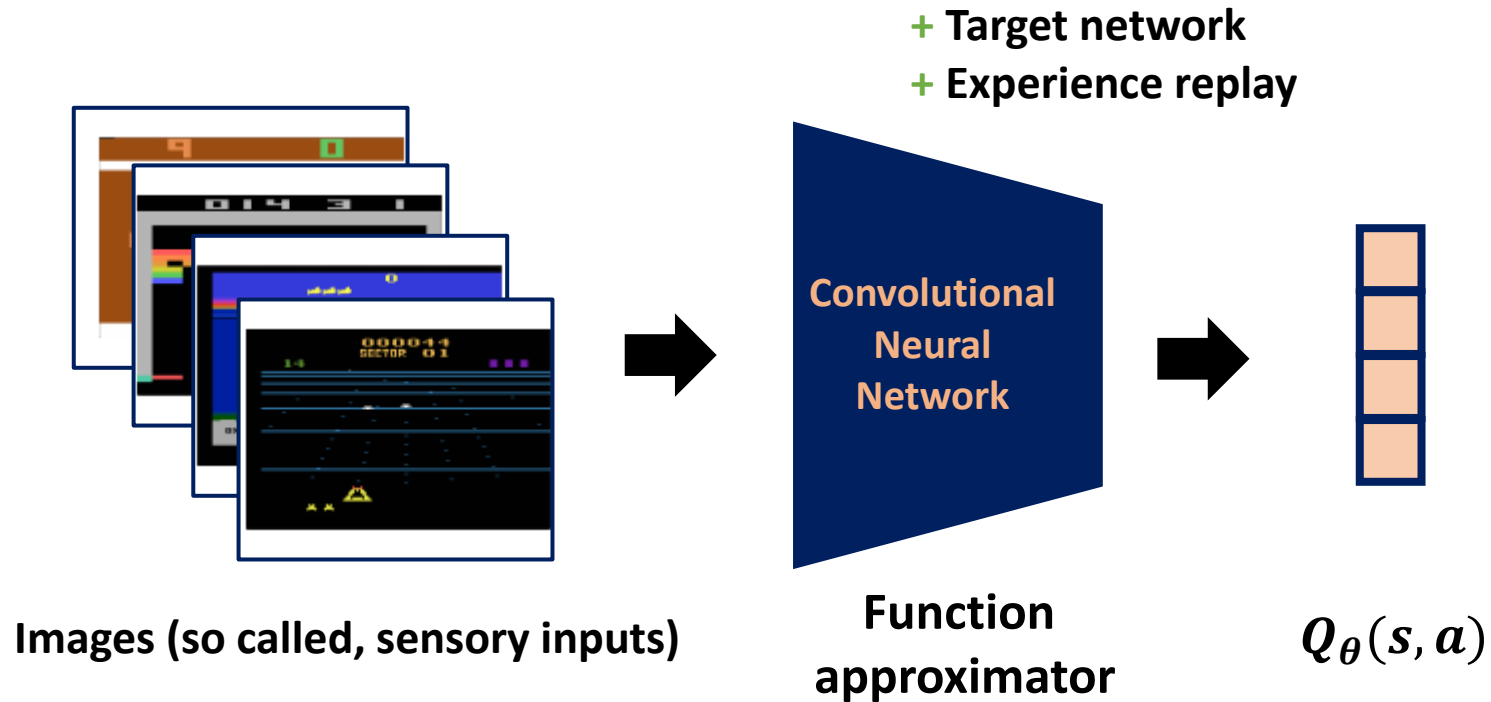
DQN (Nature version)



Feb 26, 2015

- DQN is the “first” deep learning model to successfully learn control policies “directly” from high-dimensional sensory input using reinforcement learning.
- *Not current (2020) SOTA*, but DQN papers proposed **important tricks** for stabilizing the training of deep neural network based RL methods.

Introduction



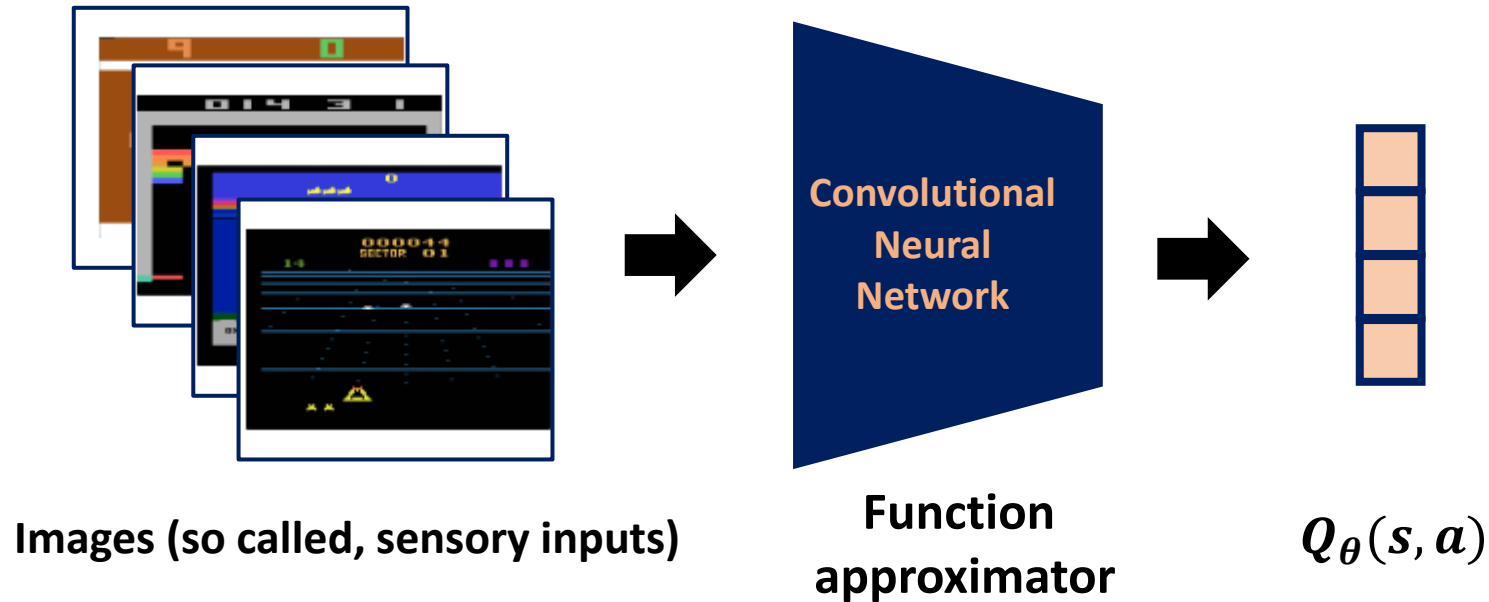
- Overcoming “curse of dimensionality” with function approximators.
- Tricks for stabilizing training procedures:
 - Experience replay *
 - Target network **

* Long-Ji Lin. Reinforcement learning for robots using neural networks. Technical report, DTIC Document, 1993

** Firstly proposed from the 2nd DQN paper.

*** Game images are reproduced from the DQN paper <Playing Atari with Deep RL>

Neural network as a value function approximator



The idea of using (deep) neural network as a value function approximator has been proposed [1,2, 3] before DQN.

- However, the non-linear methods, including neural, suffers from the training instability even in low state-action space.
- To overcome the instability, numerous objective-by-objective methods were proposed.

[1] Neural Fitted Q Iteration - First Experiences with a Data Efficient Neural Reinforcement Learning Method (2005)

[2] Q-Learning in Continuous State and Action Spaces (1999)

[3] Reinforcement learning for robots using neural networks (1993)

- DQN mitigate three issues that is regards to using non-linear function approximator in RL.
 1. “Curse of dimensionality” with Convolutional Neural Network
 2. “Temporal dependency among the samples” with Experience replay
 3. “Moving target issue” with Target network

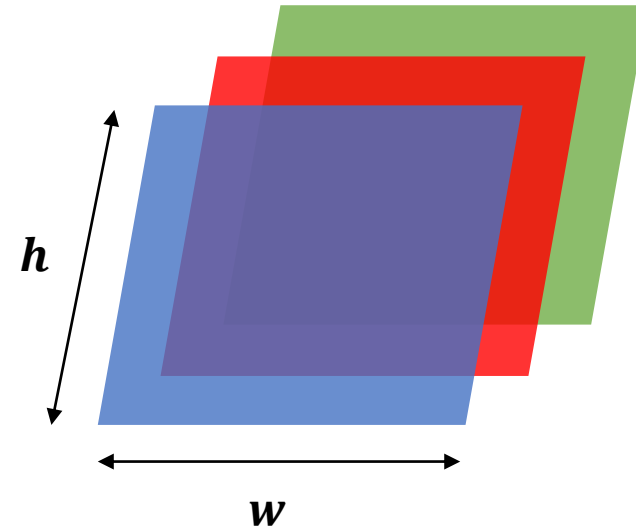
1. Curse of dimensionality

<Image source: <http://blog.comart.io/posts/opencv-rgb-1>>



8-bit RGB image with width w and height h

Each cell has a integer value between $(0, 255 = 2^8)$



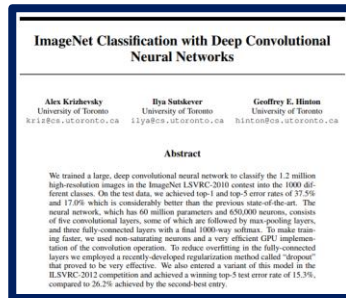
Tensor representation of image $\in \mathbb{R}^{3 \times w \times h}$

That is, entire cardinality of state set $\mathcal{S} = (2^8)^{3wh}$

In DQN paper, $w = 210$ and $h = 160$. $|\mathcal{S}| = 2.58 \times 10^7$

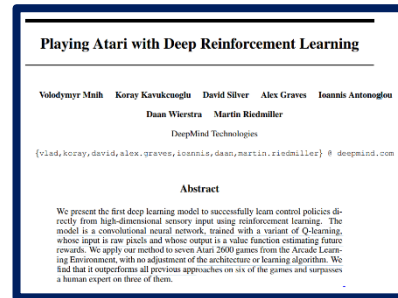
Mitigate “Curse of dimensionality” with CNN

Alexnet (NIPS 2012)



Dec 3, 2012

DQN (Arxiv version)



Dec 19, 2013

DQN (Nature version)



Feb 26, 2015

What is Alexnet? and what happened in 2012?

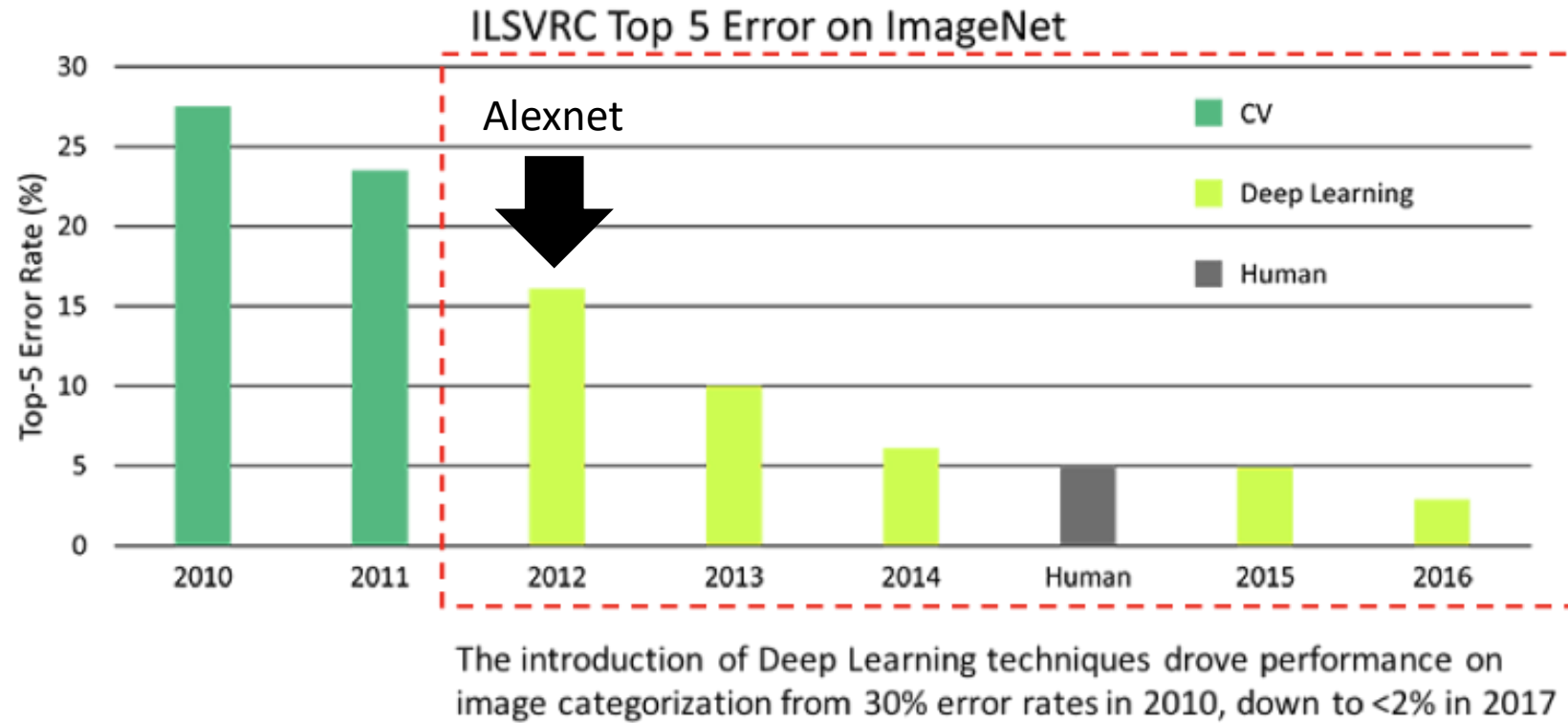
ImageNet Challenge



- 1,000 object classes (categories).
- Images:
 - 1.2 M train
 - 100k test.



Beyond the human performance



“Alexnet” was the king of image processing

<Image source: ImageNet Classification with Deep Convolutional Neural Networks>

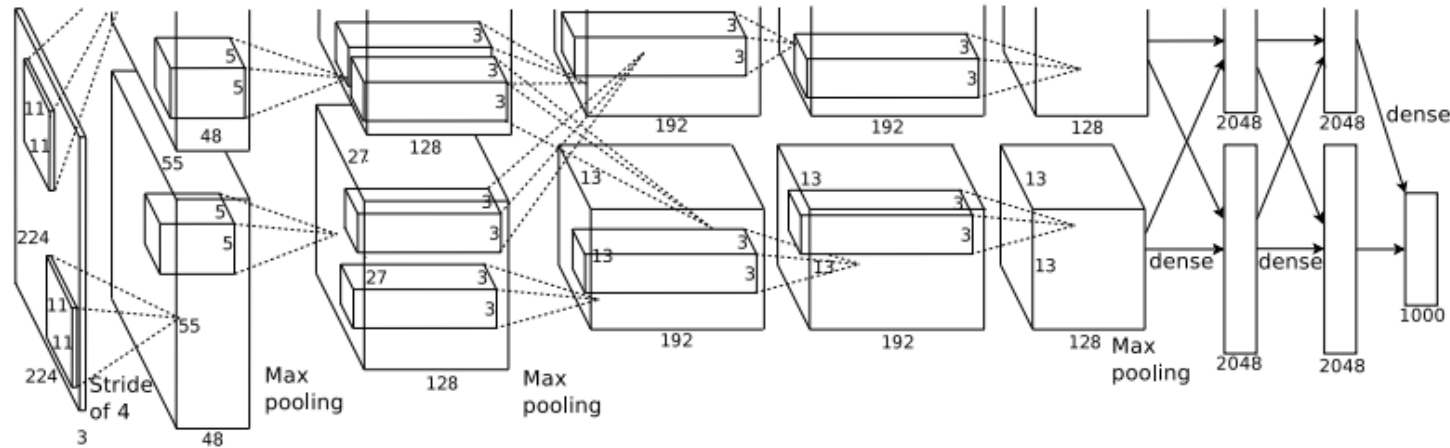
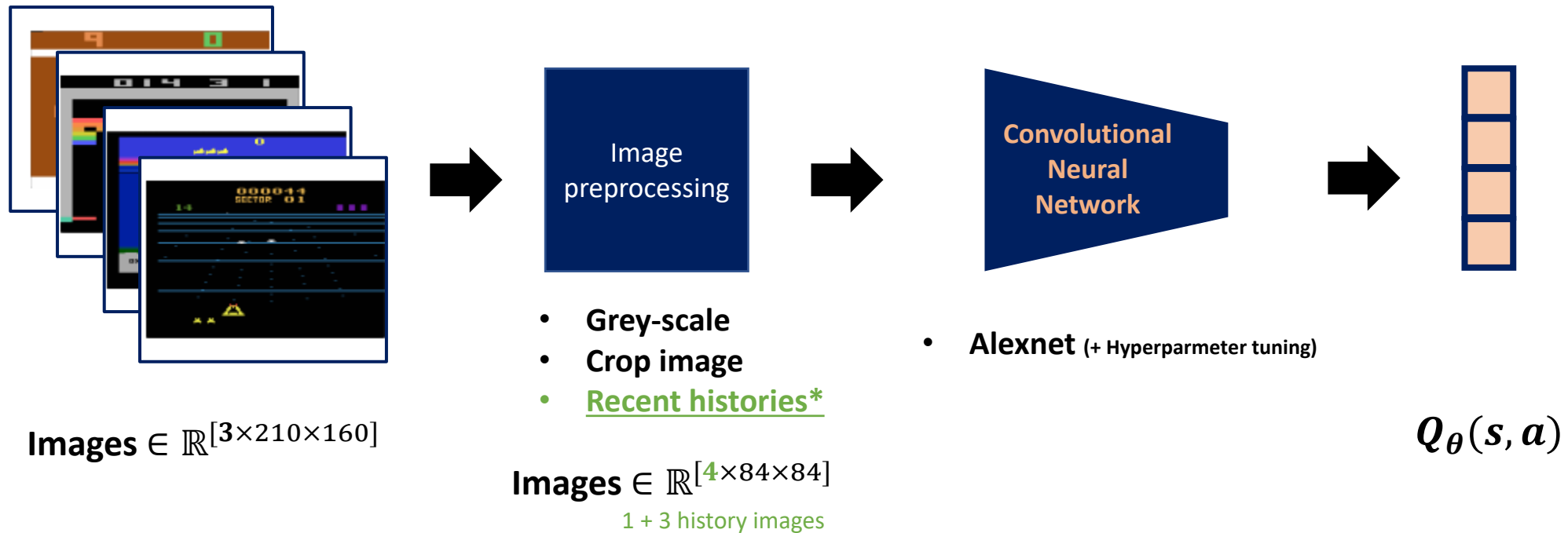


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network’s input is 150,528-dimensional, and the number of neurons in the network’s remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

- Alexnet is a grandfather of modern CNNs. It is empirically confirmed that the ‘hidden vectors (aka embedding) in “deeper” layer contains condensed information about the input image’
- That is, we can represent the input images with small dimensional tensors.

Forward pipeline of DQN



Recent histories* is the recent histories (images) of current input image.

- Both of DQN paper do not report the effect of using/not using history extensively.
- However, “using histories” is now considered as one of core trick to improve RL algorithm performance.

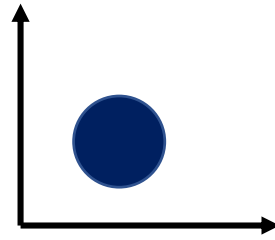
Why history is helpful?

$$s_t = [x_t, y_t]$$

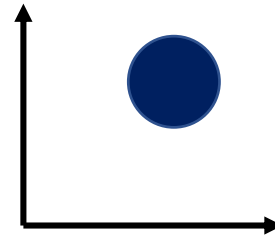
where x_t and y_t are the x and y coordinates of ball at time t .

$$a_t = [F_t^x, F_t^y]$$

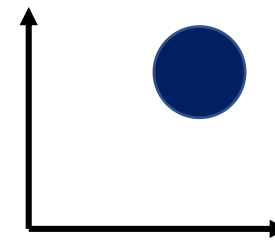
where F_t^x and F_t^y are the x and y directional force on ball at time t .



$$s_0 = [x_0, y_0]$$
$$a_0 = [F_0^x, F_0^y]$$



$$s_1 = [x_1, y_1]$$
$$a_1 = [F_1^x, F_1^y]$$



$$s_2 = [x_2, y_2]$$
$$a_2 = [F_2^x, F_2^y]$$

- From state s_t , we **cannot** infer velocity and acceleration of the ball.
 - If we have histories, then we **can** approximate velocity and acceleration of the ball.
-
- Sometimes, the states do not contain enough information such that the corresponding MDP becomes really Markovian.
 - Especially, in high dimensional setting, It is very hard to check the state action pair is actually sufficient statistics to predict next state.
 - Concatenating histories to the original state is a well defined way to define MDP from non-MDP. [1]

2. Temporal dependency among the samples

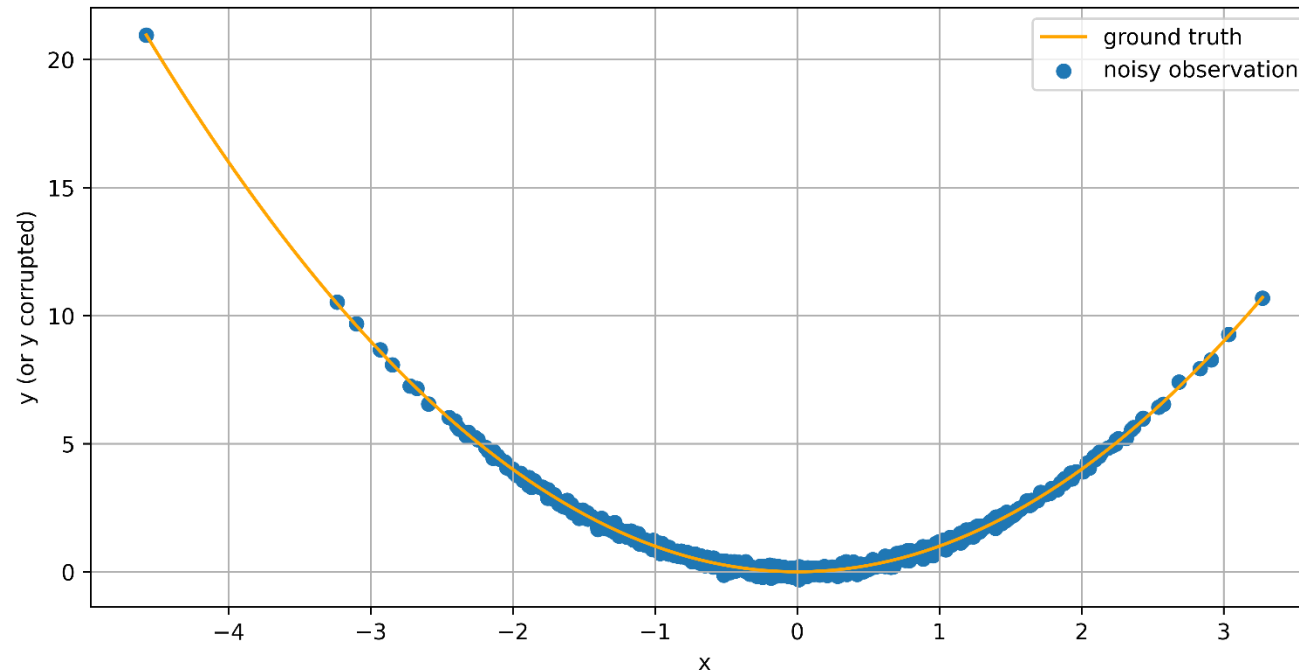
$$Q(s, a) \leftarrow Q(s, a) + \eta \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

Update the Q estimate whenever the new experience is achieved.

- (On-line) Q-Learning innately has instabilities while estimating the state-action value $Q(s, a)$
 - In the early phase of learning, estimation of $Q(s, a)$ as well as $Q(s', a')$ are inaccurate.
 - $\max_{a'} Q(s', a')$ can make the Q-Learning algorithm to overestimate the state-action values. (aka, Maximization bias of Q-Learning)
 - ϵ greedy policy based on $Q(s, a)$ can change **drastically** after single update.
 - Due to the sequential nature of MDP, the recent experiences are always correlated.

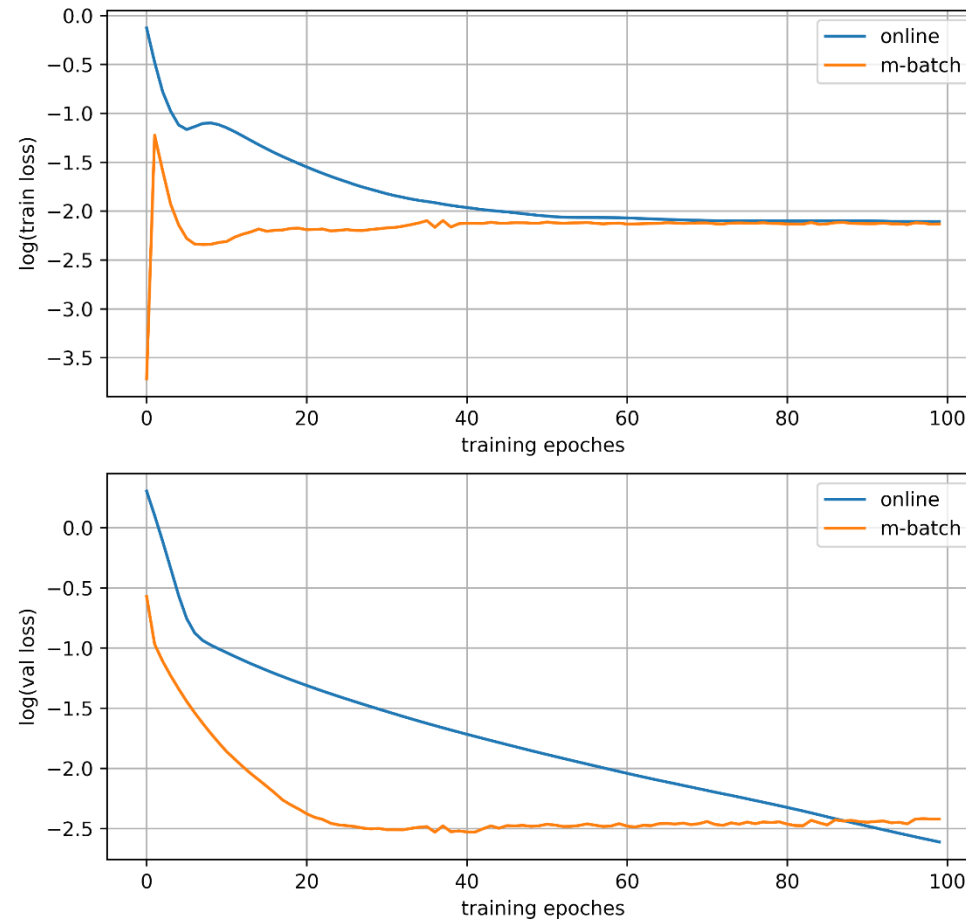
→ Requires to use “(mini) batch” update $Q(s, a)$
- DQN papers suggest to use “experience replay”
 - It is noteworthy that “experience replay” which was firstly proposed by [1]

Online update vs. mini batched update



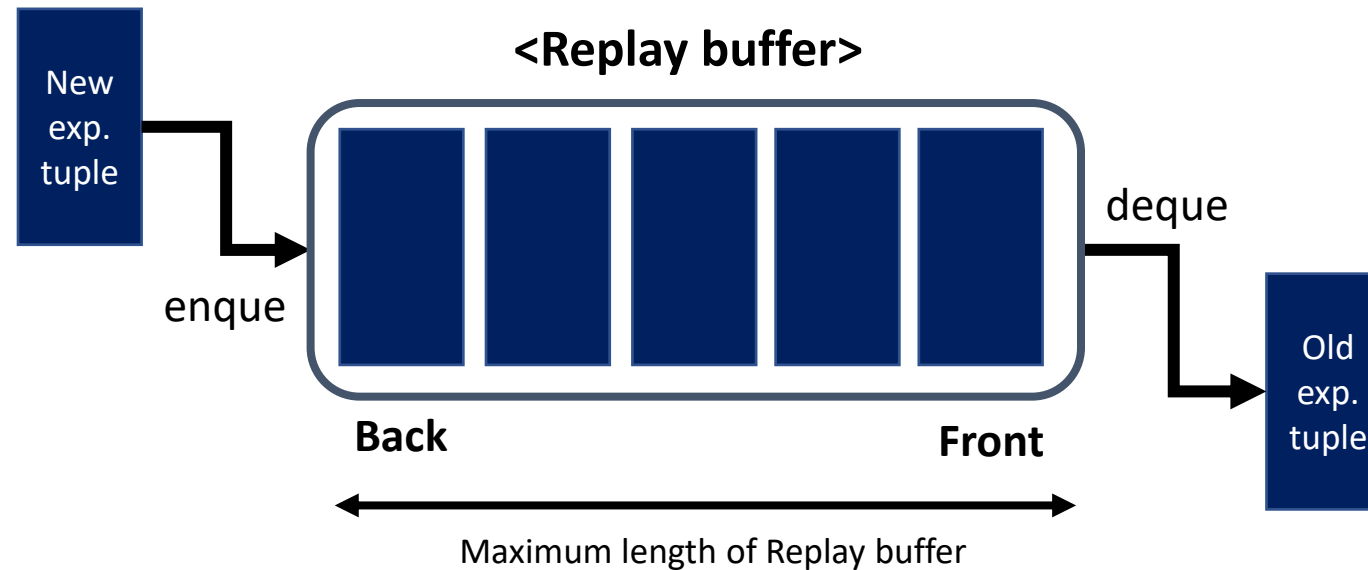
- Object: train 2 layer ReLU network predicting $y = x^2$ with stochastic gradient descent algorithm
 - The training loss is MSE
- We can only observe corrupted version of $y' = y + \epsilon$ where $\epsilon \sim \mathcal{N}(0, 1^{-3})$.
 - training data pairs are $\{x_i, y'_i\}_{i=1 \dots n}$ and validation data pairs are $\{x_i, y'_i\}_{i=1 \dots n}$
- Consider two different training scheme
 1. Online-like update (single sample is used for estimate losses and updating gradient)
 2. Mini-batched update ($m > 1$ samples are used for estimate losses and updating gradient)

Online update vs. mini batched update



- Online-like update case achieved smaller training error than mini-batch update case.
- However, Online-like update achieved significantly larger validation error than mini-batch update.

Experience replay



- **Experience replay** is a method that allows to update RL agent with the batch of experiences.
- Replay buffer (often called as memory, buffer) $\mathcal{D}$ is a finite length queue that stores the recent experiences tuples (s, a, r, s')
- During training DQN, the experiences will be enqueued to the R.B..
- Queue has First-In-First-Out (FIFO) structure. Therefore, order experience will be discarded first.

Optimize Q values with Experience replay

<Online>

$$Q(s, a) \leftarrow Q(s, a) + \eta \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

<Function approximation>

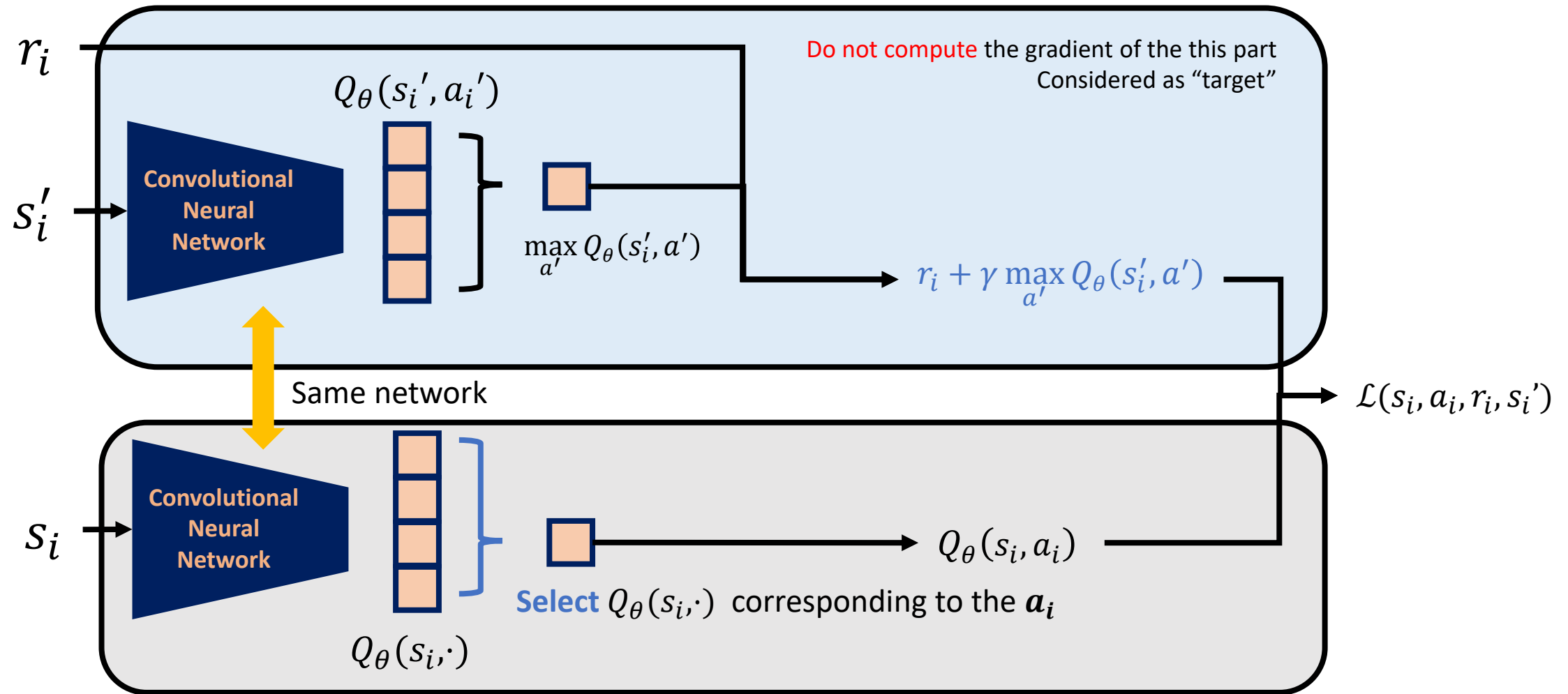
$$\theta \leftarrow \theta + \eta \frac{\partial \mathcal{L}(s, a, r, s')}{\partial \theta}$$
$$\mathcal{L}(s, a, r, s') = \left\| r + \gamma \max_{a'} Q_{\theta}(s', a') - Q_{\theta}(s, a) \right\|_2$$

<Function approximation + Experience replay>

$$\theta \leftarrow \theta + \eta \frac{\partial \frac{1}{m} \sum_{i=1}^m \mathcal{L}(s_i, a_i, r_i, s_i')}{\partial \theta}$$
$$\mathcal{L}(s_i, a_i, r_i, s_i') = \left\| r_i + \gamma \max_{a'} Q_{\theta}(s_i', a') - Q_{\theta}(s_i, a_i) \right\|_2$$
$$(s_i, a_i, r_i, s_i') \sim \mathcal{D}$$

Experience tuples are **randomly** samples from the buffer $\mathcal{D}$

Implementing Bootstrapping method on deep neural network



$$\text{Loss function: } \mathcal{L}(s_i, a_i, r_i, s'_i) = \left\| r_i + \gamma \max_{a'} Q_\theta(s'_i, a') - Q_\theta(s_i, a_i) \right\|_2$$

Remarks on Experience replay

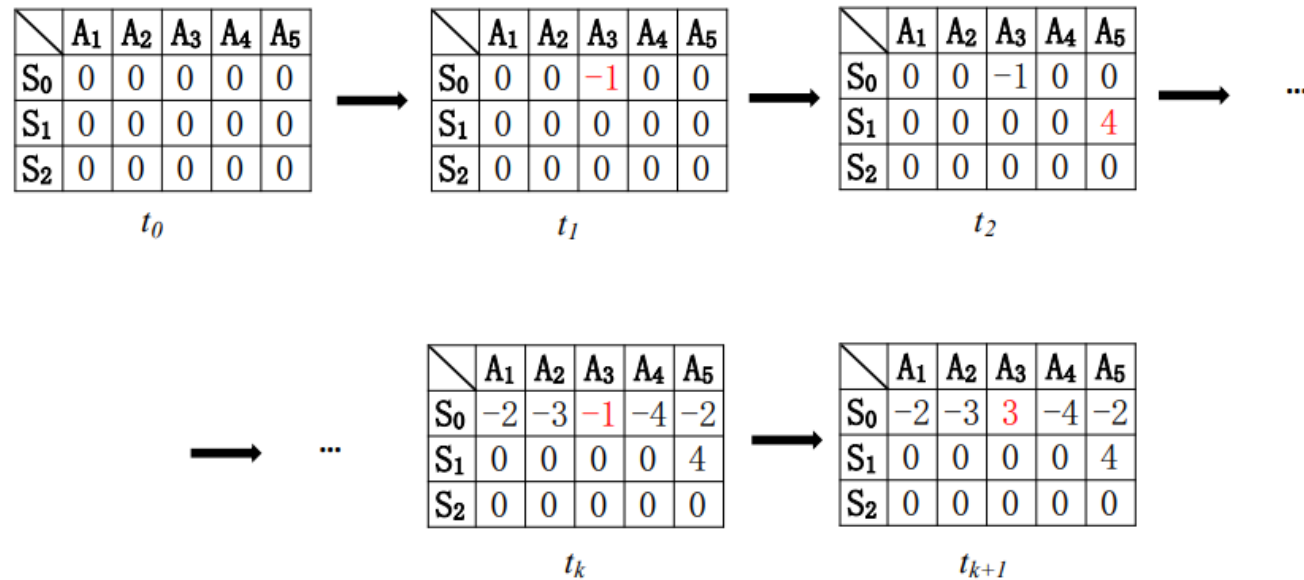
- The vanilla experience replay is applicable only to off-policy method.
- It is still controversial that the effect of batch size to the generalization performance of deep neural networks.

3. Moving target issue

Q-learning **w/o** function approximators,

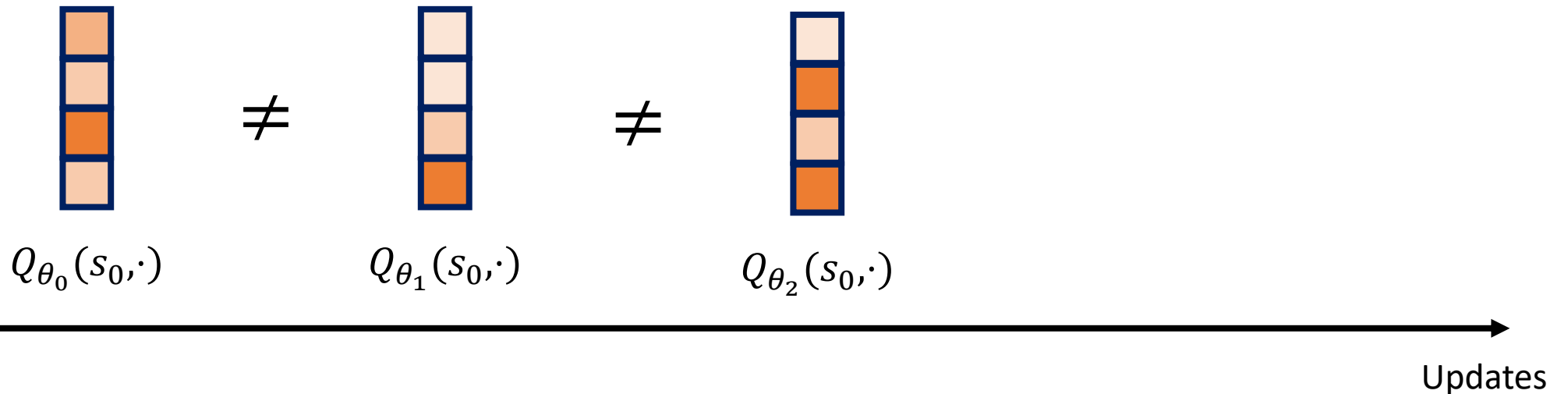
$$Q(s, a) \leftarrow Q(s, a) + \eta \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

The update of specific $Q(s, a)$ doesn't change the estimate of $Q(s, a)$ of other state-action pairs.



3. Moving target issue

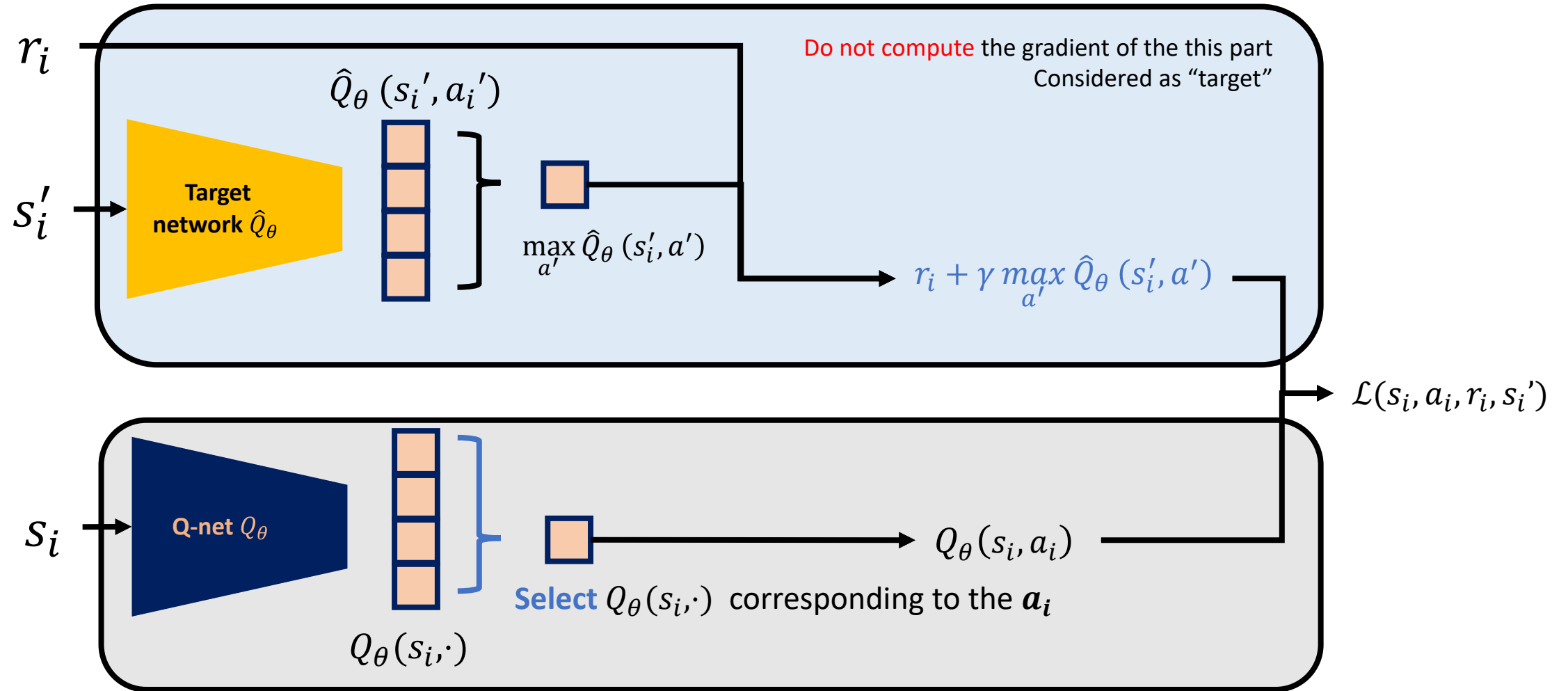
- In the case of Q-learning **with** function approximators, the single update of Q values may change the Q values different state-action pairs.
- The degree of changing becomes severe when we use flexible function approximators such as neural networks.



Target network

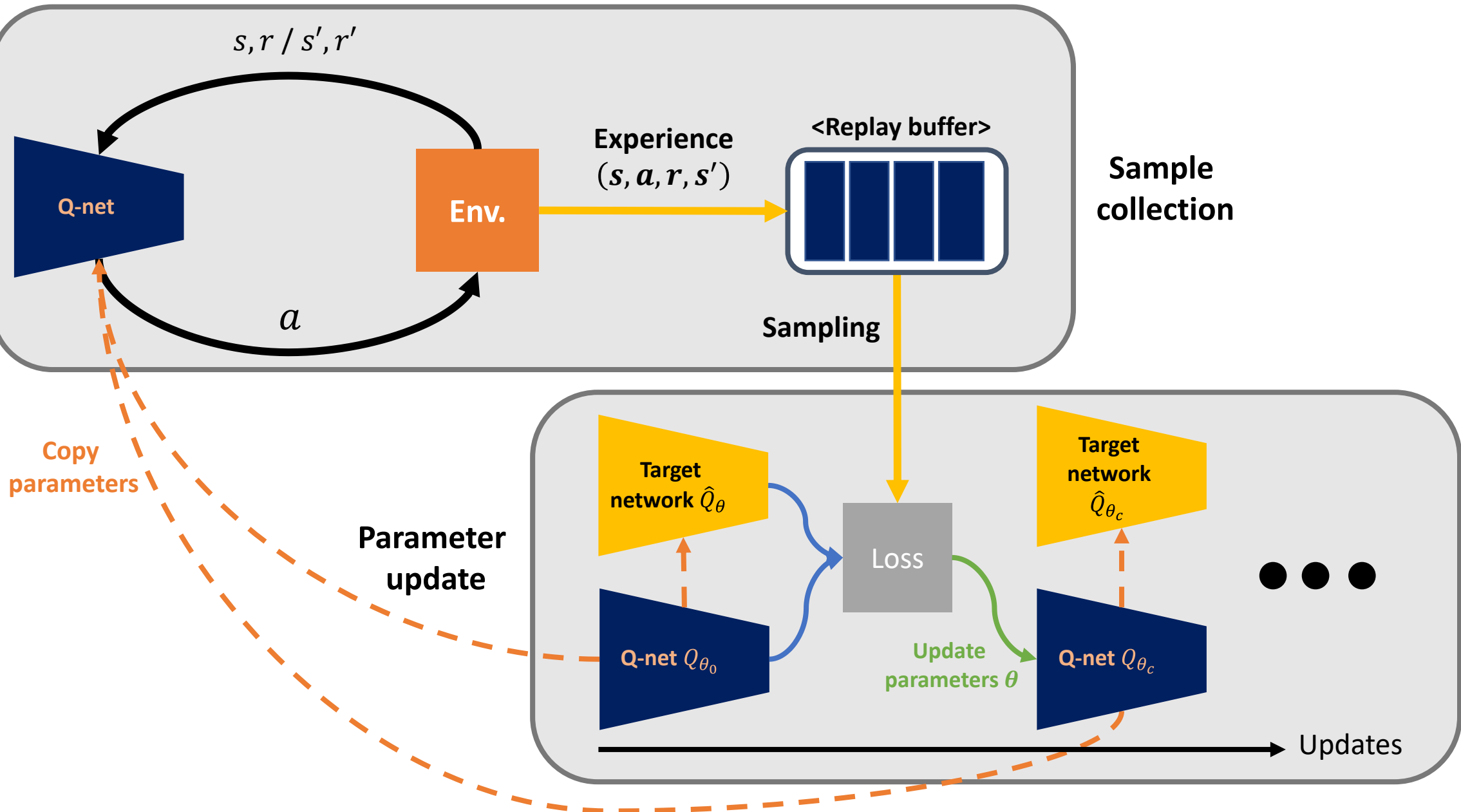
- DQN (nature version) propose to use “Target network”
- The “target network” $\hat{Q}_\theta$ is a copy of main Q-network Q_θ .
- During the given update span C , use the target network to compute Q-targets.
- Update the main Q-network
- After C updates, Copy the parameters of Q_θ and apply the parameters to $\hat{Q}_\theta$.

Update Q network with Target network



Loss function: $\mathcal{L}(s_i, a_i, r_i, s'_i) = \left\| r_i + \gamma \max_{a'} \hat{Q}_\theta(s'_i, a') - Q_\theta(s_i, a_i) \right\|_2^2$

Schematic of DQN training



Put everything together

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

 With probability ε select a random action a_t

 otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

 Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

 Every C steps reset $\hat{Q} = Q$

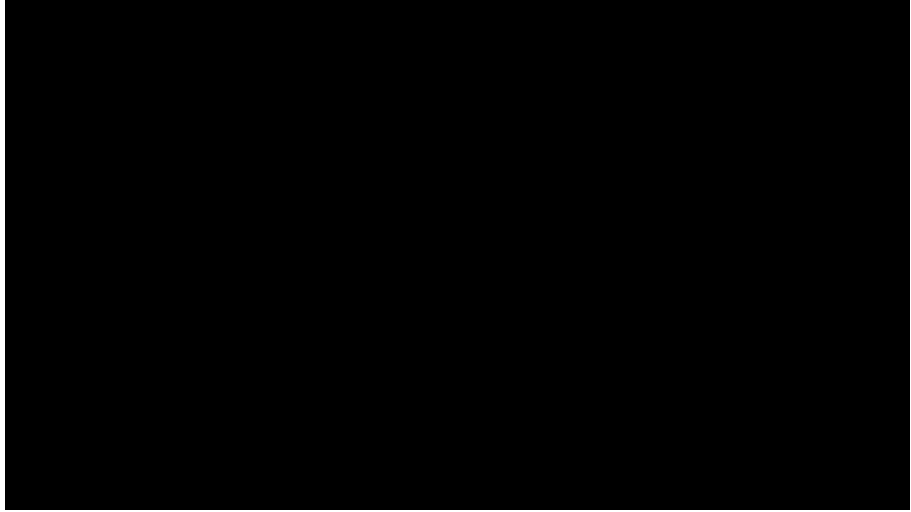
End For

End For

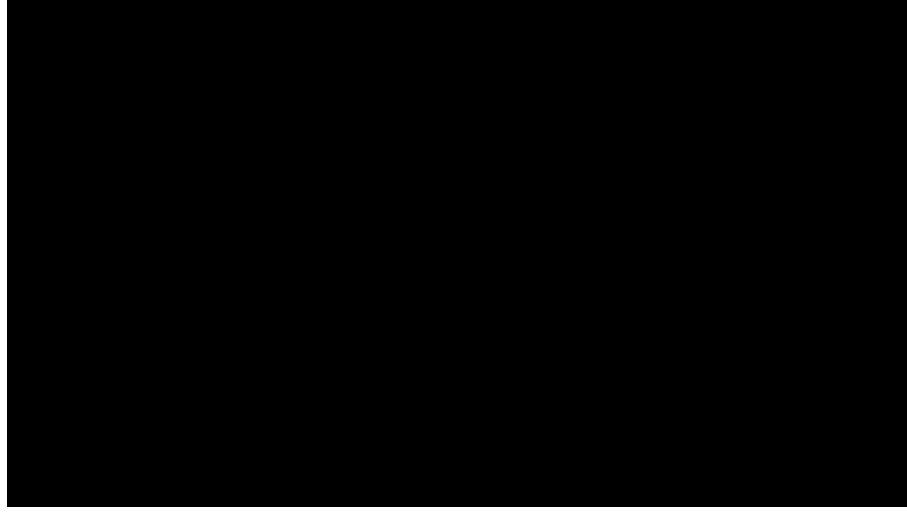
Remarks on Target network

- DQN-style target network update is now often called as “hard” update.
- In principle, The hard update scheme encounter moving Q-target issues on every update span C .
- “soft” target update also has been proposed from DDPG paper [1].
 - $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$
 - θ' is the updated parameter and θ is the old parameter and the lagging parameter $\tau \in [0,1)$
 - When $\tau = 0$, the “soft” target update becomes “hard” update with $C = 1$.
 - When $\tau = 1$, the “soft” target update becomes non-updating.

Performance evaluation



<Breakout>



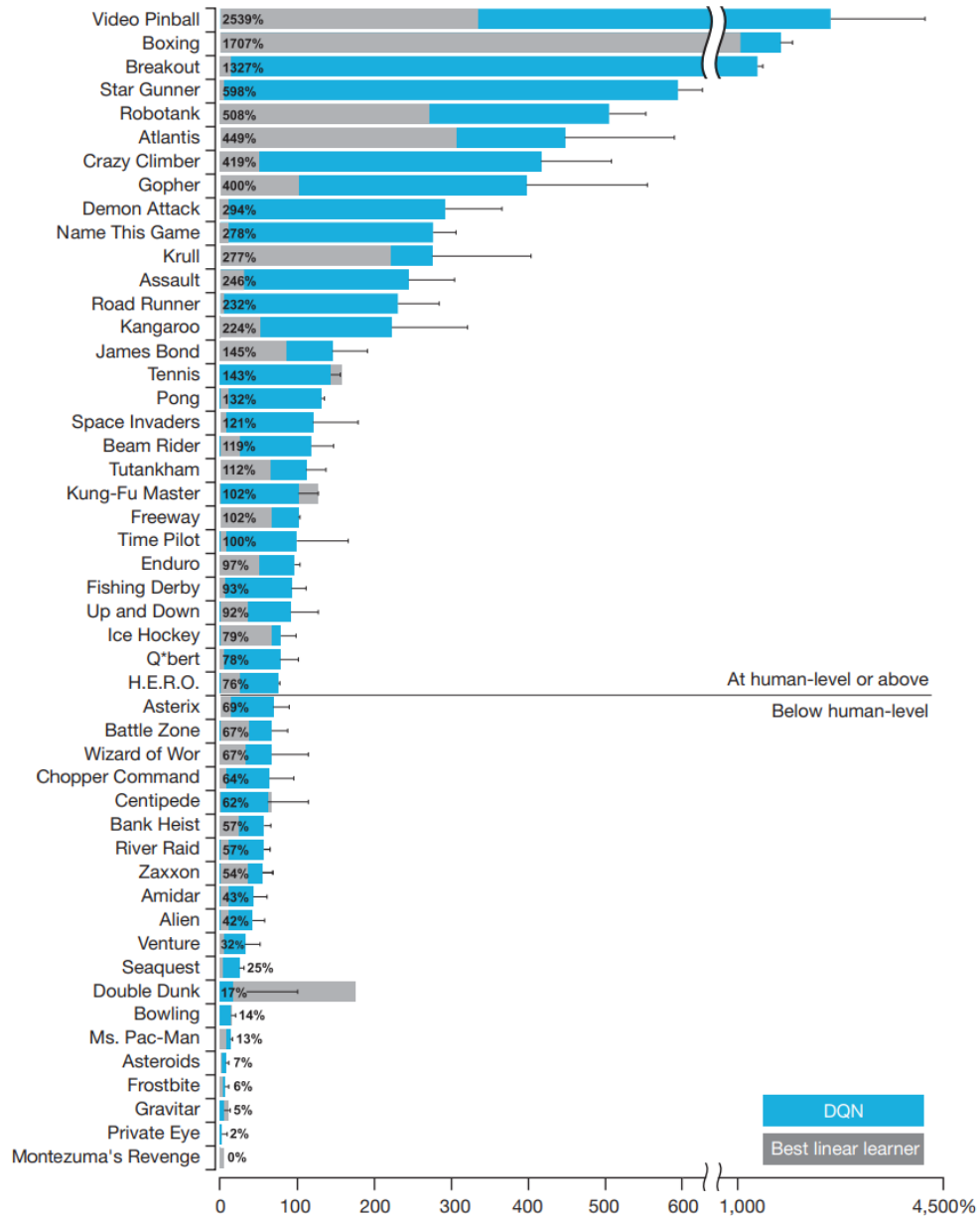
<Boxing>

- DQN is the first RL model that can solve various “**visual control problems**” with the same hyperparameters
- The evaluation environments are classic **Atari 2600 games**
 - Why games are used as the environments?
 1. Easy to collect experiences
 2. Reward is well defined. (Almost) every environment step, the RL agent can get meaningful rewards

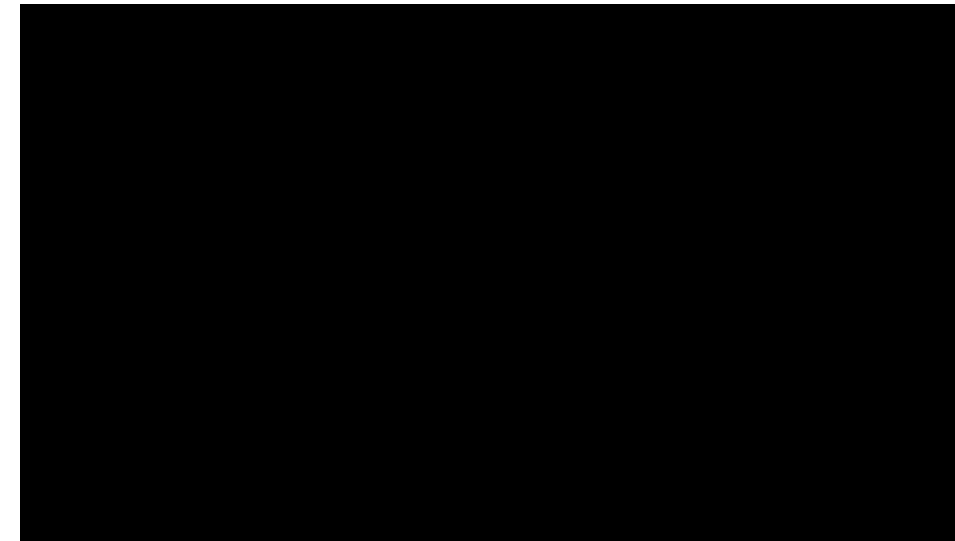
RL with the sparse reward is still challenging!

Performance evaluation

<Figure source: Human-level control through deep reinforcement learning>



DQN: Breakout



DQN: Space invaders

Ablation study

This table is reproduced from the source: Human-level control through deep reinforcement learning.

Game	W/ ER W/ target Q	W/ ER W/O target Q	W/O ER W target Q	W/O ER W/O target Q
Breakout	316.8 (x99.0)	240.7 (x75.21)	10.2 (x3.18)	3.2 (x1)
Enduro	1006.3 (x34.58)	831.4 (x28.57)	141.9 (x4.87)	29.1 (x1)
River Raid	7446.6 (x5.12)	4102.8 (x2.82)	2867.7 (x1.97)	1453.0 (x1)
Seaquest	2894.4 (x10.49)	822.6 (x2.98)	1003.0 (x3.63)	275.8 (x1)
Space Invader	1088.9 (x3.61)	826.3 (x2.74)	373.2 (x1.23)	302.0 (x1)

* Higher is better. **Values in parenthesis indicates normalized values toward naïve Q network models

- Experience replay (ER) and target Q network are helpful for solving various Atari games.
- The effectiveness of ER and Target Q were varying across the games.
- DQNs experience “10 million frames” for each game and setup → Still notorious sample inefficiency.

Summary

- DQN revolutionize deep RL field (2013)
- The technical challenge were:
 1. High dimensional state-action spaces
 2. Temporal dependency among the samples
 3. Moving target
- The main ideas are:
 1. Using SOTA CNN architecture (Alexnet)
 2. Experience replay
 3. Target network
 4. **Using history inputs**
- The limitations are:
 1. Experience replay are only applicable to the **off-policy** method
 2. Random experience replay
 3. Hard target network
 4. Sample inefficiency (still major problem of deep RLs)